



Technical Design Document

HQ → POS Price Synchronization

Middleware integration engine for Oracle Retail Xstore

Participants: Xuan Sang, Minh Kien

This document describes how the system specified in the URD is built. It follows an industry design-document structure (arc42 / IEEE 1016 / C4 / architecture decision records) and is intended for internal engineering review. The URD remains the requirements baseline.

Document Control

Version history

Ver	Date	Author	Summary of change
1.0	28/06/2026	Xuan Sang, Minh Kien	Initial technical design.
2.0	29/06/2026	Xuan Sang, Minh Kien	Restructured to an industry layout (arc42 / C4 / IEEE); added goals, non-goals, and the requirements digest.
3.0	30/06/2026	Xuan Sang, Minh Kien	Added the C4 architecture views (system context and containers) and the architecture decision records (ADRs).
4.0	01/07/2026	Xuan Sang, Minh Kien	Added the component and domain model, the API design, and the runtime sequence flows (S1–S5).
5.0	02/07/2026	Xuan Sang, Minh Kien	Added the test plan (Section 8) and deployment view (Section 3.8); detailed the console API (Section 7.3); applied review feedback.

Table 1: Document version history.

Reviewers & approvers

Role	Name	Responsibility
Author	Xuan Sang, Minh Kien	Produce and maintain the design.
Technical reviewer	anh Thành, chị Thu	Review architecture and design decisions.
Approver	anh Duy, anh Lý	Approve for build; own open items.

Table 2: Reviewers and approvers.

Contents

1	Introduction	4
1.1	Purpose and audience	4
1.2	Scope and boundary	4
1.3	Goals and non-goals	4
1.4	Assumptions, constraints, and dependencies	5
1.5	References and standards	5
1.6	Terminology	5
1.7	Abbreviations	6
2	Requirements	7
2.1	Functional requirements (digest)	7
2.2	Non-functional requirements and targets	8
2.3	Quality-attribute scenarios	9
3	System Context and Architecture Overview	11
3.1	Business context	11
3.2	System context	11
3.3	Containers	12
3.4	Daily processing flow	16
3.5	Batch lifecycle	17
3.6	Architectural style and rationale	18
3.7	Technology choices	19
3.8	Deployment view	19
4	Architecture Decision Records	22
5	Component Design (C4 level 3)	28
5.1	Component overview	28
5.2	Intake components	28
5.3	Processing components	29
5.4	Supporting components	30
5.5	Static structure - domain model	31
5.6	Static structure - key application classes	32
5.7	Extensibility – adding a new output target	34
6	Runtime Scenarios	35
6.1	Runtime sequence – end-to-end daily sync [main success path with branch points] .	36
6.2	S1 – Intake rejection [exception flow]	38
6.3	S2 – Duplicate batch [alternative flow]	39

6.4	S3 – Partial batch [alternative flow]	40
6.5	S4 – Write failure and retry [exception flow]	41
6.6	S5 – Instance failover [exception flow]	42
7	API Design	43
7.1	Conventions	43
7.2	Intake endpoint	43
7.3	Console and administration endpoints	46
7.4	Error catalog	50
7.5	Rate limiting and replay protection	50
8	Test Plan	51
8.1	Test levels and tooling	51
8.2	Functional test cases	51
8.3	Non-functional test cases	56
8.4	Entry and exit criteria	56
A	Requirements traceability	57
B	Persistence schema and key indexes	57

1. Introduction

1.1. Purpose and audience

This Technical Design Document (TDD) describes *how* the Headquarters (HQ) to Point-of-Sale (POS) price synchronization system is built. It turns the requirements agreed in the User Requirement Document (URD) into a concrete software design - architecture, decisions, components, interfaces, data, behavior, reliability, security, deployment, and operations - so that BTM engineers can implement, review, and hand over the system with a shared understanding.

Readers are expected to know Java, Spring Boot, relational databases, Representational State Transfer (REST), and basic Oracle Retail / Xstore DataLoader concepts.

1.2. Scope and boundary

The system is a **middleware integration engine** with a **monitoring console**. Each night, HQ delivers the day's price changes as a **batch event** over an Application Programming Interface (API). The system authenticates it, validates and maps the records, builds an Oracle DataLoader maintenance (MNT) file, and writes that file into the **Xcenter inbound folder**.

Everything downstream of that folder - distribution to stores, loading and applying at the POS, and auto-revert - is performed by Xstore and is *out of scope*. The design therefore stops at the Xcenter inbound folder; that boundary is the same one asserted in the URD.

1.3. Goals and non-goals

Goals.

- Convert each nightly HQ price batch into a well-formed DataLoader MNT file and write it to the Xcenter inbound folder before the daily cut-off.
- Only validated, complete files are produced; Xstore never ingests a malformed or partial file.
- Lose no event: if Xcenter is unreachable or a write fails, the event is retried until it succeeds or is escalated.
- Be idempotent: re-sending the same batch, or a superseding version of a change, never produces duplicate or incorrect output.
- Keep a queryable audit trail of every event and pipeline step, retained for at least two months.
- Keep the intake endpoint and processing highly available.
- Allow Ops to change mapping, scope, and connection settings through the console without code changes or redeployment.
- Provide a monitoring console (dashboard, event history, logs) for Ops.

Non-goals.

- Distributing files from Xcenter to stores, applying prices at the POS, or auto-reverting - all handled by Xstore.
- Authoring or deciding prices; the system only transforms what HQ sends.

- Real-time or sub-minute propagation; processing is a nightly batch.
- Managing Xstore / Xcenter configuration, or being HQ's system of record.

1.4. Assumptions, constraints, and dependencies

Assumptions. HQ delivers one price batch per night over the API; records may carry effective dates up to about one week ahead (URD functional requirement FR-10), and a record with no effective date defaults to the next day, D+1 (FR-08).

Prices are typically scoped by zone/region (not always per individual store); the Xcenter inbound folder is reachable on the local network; the nightly window is sufficient to process and write one batch.

Constraints. On-premises deployment co-located with Xcenter; the Oracle DataLoader file format is fixed and external to this system; the implementation stack is Java 17 / Spring Boot 3.x / PostgreSQL; the document targets internal BTM review.

Dependencies. The MNT record layout and file naming follow the Oracle Xstore Host Interface Guide and are externalised in the mapping profile (Section 4, architecture decision record ADR-08); the HQ JavaScript Object Notation (JSON) contract is the envelope/record structure documented in the API design (Section 7).

Alert channels and the partial-vs-abort threshold are decided in this document (Section 4 ADR-07, and the Alerting component) and held in configuration.

1.5. References and standards

- User Requirement Document (URD) - HQ-to-POS Price Synchronization (requirements baseline).
- Oracle Retail Xstore Host Interface Guide / DataLoader specification (authoritative for the MNT layout).
- arc42 architecture documentation template; IEEE 1016 Software Design Descriptions; the C4 model for software architecture diagrams; AWS Well-Architected and ISO/IEC 25010 quality attributes for the non-functional design; OWASP / STRIDE for the threat model.

1.6. Terminology

- **Engine** - the headless service that receives, validates, transforms, and writes files.
- **Console** - the engine's web UI: configure connections and mapping, and monitor the pipeline (monitoring only; it never authors or sends prices).
- **Event / batch** - one nightly price message from HQ, carrying many records.
- **Record** - one item/location price line within a batch; it becomes one MNT detail row.
- **Set-aside / partial** - a record that fails validation or mapping is set aside and flagged; the remaining valid records still produce a file (status PARTIAL).
- **Xcenter inbound folder** - the network destination where the engine writes the MNT file; the output boundary of this system.

- **Idempotency** - processing the same input twice yields the same single result, with no duplicate output.

1.7. Abbreviations

Each abbreviation is expanded at its first use in the text; this table collects them for quick reference.

Abbr.	Full form	Abbr.	Full form
CRUD	Create, Read, Update, Delete	HMAC	Hash-based Message Auth. Code
HTTP(S)	HyperText Transfer Protocol	JDBC	Java Database Connectivity
JSON	JavaScript Object Notation	LOC_TYPE	Xstore location-type code
POSIX	Portable OS Interface	REST	Representational State Transfer
SFTP	SSH File Transfer Protocol	SMB/CIFS	Server Message Block (share)
ADR	Architecture Decision Record	API	Application Programming Interface
JPA	Jakarta Persistence API	LTS	Long-Term Support
MNT	DataLoader maintenance file	MVC	Model-View-Controller
NFR	Non-Functional Requirement	NFS	Network File System
POS	Point of Sale	SKU	Stock Keeping Unit (item code)
SPA	Single-Page Application	SQL	Structured Query Language
TDD	Technical Design Document	TLS	Transport Layer Security
TTL	Time To Live (cache lifetime)	URD	User Requirement Document
XML	Extensible Markup Language	C4	Context/Container/Component/Code
CI	Continuous Integration	FR	Functional Requirement
HA	High Availability	HQ	Headquarters (pricing system)
IP	Internet Protocol (address)	TC	Test Case (Section 8)

Table 3: Abbreviations used in this document.

2. Requirements

This section restates the URD requirements at the level the design must satisfy and attaches measurable non-functional requirement (NFR) targets and quality-attribute scenarios. The URD remains authoritative for the full requirement text.

2.1. Functional requirements (digest)

The eighteen functional requirements group into a small number of capabilities:

- *Intake* - FR-01 (receive JSON delta via API), FR-04 (process by delta, not full load).
- *Security* - FR-02 (API key + Hash-based Message Authentication Code (HMAC) + Internet Protocol (IP) allowlist).
- *Validation* - FR-03 (validate before producing a file).
- *Mapping and scope* - FR-05 (map to scope and to Xstore codes; flag unmappable records).
- *File generation* - FR-06 (DataLoader MNT with correct trailer), FR-07 (carry effective dates), FR-08 (default D+1 when absent), FR-09 (write to the Xcenter inbound folder), FR-10 (daily flow, prices sent ahead).
- *Tracking and reliability* - FR-11 (track pipeline state), FR-12 (retry-on-write-failure, no loss), FR-13 (idempotency, latest version wins).
- *Operations* - FR-14 (end-to-end audit log), FR-15 (monitoring dashboard), FR-16 (event history), FR-17/FR-18 (alert Ops via dashboard and push channels).

2.2. Non-functional requirements and targets

ID	Attribute	Design target (measurable)
NFR-01	Scale	Process one nightly batch covering up to 250 stores / 75 000+ items and write the file within the daily cut-off window.
NFR-02	Security	HTTPS / Transport Layer Security (TLS) 1.2+; API key + HMAC-SHA256 + IP allowlist on intake; no secret stored in plaintext (ADR-11, ADR-12).
NFR-03	Data integrity	Every produced file is complete with a correct trailer count; Xstore ingests only complete files (pickup contract, ADR-05).
NFR-04	Reliability	On write failure, bounded retries with increasing backoff, then alert; 0 events lost across restarts.
NFR-05	Consistency	The written file completely and correctly reflects the validated records of the batch.
NFR-06	Availability	Intake and processing stay available; single-instance failure does not stop the nightly run.
NFR-07	Observability	Per-event pipeline status, structured logs, dashboard, and alerts.
NFR-08	Auditability	Event and log records queryable and retained ≥ 2 months.
NFR-09	Extensibility	Add stores/regions or change mapping with configuration only - no core-code change.

Table 4: Non-functional requirements and their measurable design targets.

2.3. Quality-attribute scenarios

These concrete scenarios make the NFRs testable (stimulus → response → measure).

Attribute	Scenario (stimulus → response)	Measure
Performance	The nightly batch arrives → the system validates, maps, builds, and writes the MNT.	Completes within the cut-off window for a full-catalog batch.
Reliability	Xcenter is unreachable for several hours when the batch is ready → the event is queued and retried with backoff.	0 events lost; the file is written once Xcenter is reachable; Ops alerted on retry exhaustion.
Availability	One application instance fails mid-run → the other instance continues; intake stays reachable.	Nightly run still completes; no double-written file.
Security	A request arrives with a missing/invalid signature or from a non-allowlisted IP → it is rejected before any processing.	401/403 returned; nothing persisted or written.
Scalability	The catalog grows toward 75 000+ items / 250 stores → the batch is still processed within the window.	Linear, within cut-off; no redesign needed.
Modifiability	A new region must be priced → Ops adds it via the console.	Configuration change only; no code change or redeploy.

Table 5: Quality-attribute scenarios: stimulus, response, and measure.

Performance budget. At the NFR-01 ceiling (250 stores / 75 000+ items) the batch is one JSON envelope of tens of thousands of records and the output is one MNT of a few MB, built by streaming rows to a local temporary file (bounded memory). The dominant cost is the single sequential write to the Xcenter share, not CPU; validation and mapping are linear, per-record, in-process work. A run of this size completes in well under the nightly window (batch generated ~22:00, cut-off 05:00 local ⇒ ~7 h available), leaving ample margin for the bounded write-retry budget. The default operational parameters below are configuration; they are chosen so that each timeout

sits safely above the real processing time (seconds-to-minutes).

Parameter	Default	Why
Nightly cut-off	05:00 local	File in place hours before stores open (around 08:00).
Dispatcher / Retry poll interval	10 s	Sub-minute pickup; negligible DB load for a nightly batch.
Lease timeout (heartbeat)	5 min (renew every 1 min)	Much longer than real processing time, so a live worker is never pre-empted; short enough to recover a dead one quickly.
Write retry	6 attempts, exponential backoff 30 s doubling, capped 10 min (about 30 min total)	Rides out transient share/network faults; then FAILED + alert for a real outage.
Replay clock-skew window	plus or minus 5 min	Bounds replay while tolerating clock drift (Section 7.5).
Audit / data retention	at least 2 months	NFR-08; window exceeds any plausible HQ re-send delay.

Table 6: Default operational parameters (configurable).

3. System Context and Architecture Overview

3.1. Business context

HQ owns pricing and produces the day's price changes. The integrator (this system) is the bridge that turns those changes into the file format Oracle Xstore expects and places it where Xstore looks for it.

Xstore then distributes to stores and applies at the POS. The integrator never reaches into HQ or into Xstore; it consumes an API call from HQ and produces a file for Xstore.

BTM builds and delivers the integrator, including its console; after handover, the customer's own operations / support staff use the console to configure the field mapping and connections and to monitor the pipeline. Running and monitoring the system in production is the customer's responsibility, not BTM's.

3.2. System context

Figure 1 shows the system as a single box among its users and neighbouring systems.

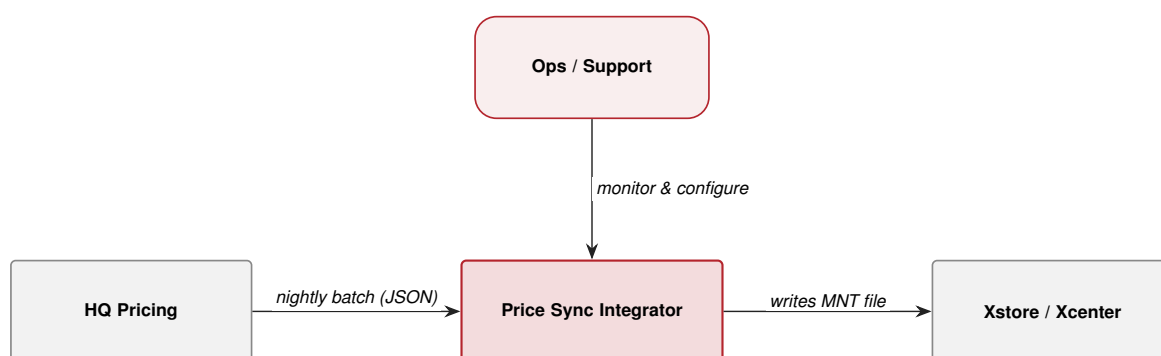


Figure 1: C4 level 1 - system context.

Flows. Each flow below is detailed in its own table.

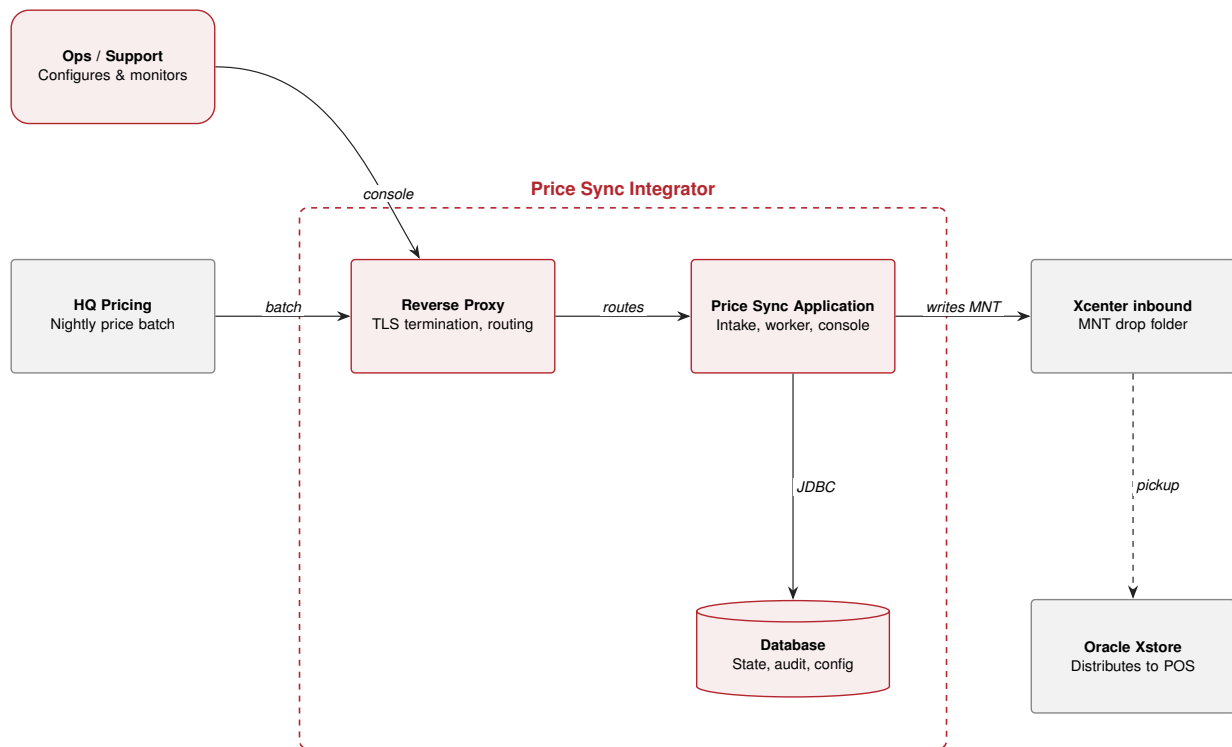
Flow 1 — HQ Pricing → Price Sync Integrator	
Source	HQ Pricing (external pricing system)
Target	Price Sync Integrator (this system)
Purpose	HQ delivers the day's price changes (delta) for processing into the format Oracle Xstore expects.
Protocol / tech	HTTPS (REST API call)
Data / payload	Nightly price batch in JSON: an envelope with an array of price-change records (item, location/zone, price, effective dates, change type).
Interaction & frequency	Asynchronous — the system accepts the batch (HTTP 202) and processes it in the background. Once per night, with prices typically dated about a week ahead.
Security	API key + HMAC-SHA256 signature + IP allowlist, all over TLS.

Flow 2 — Price Sync Integrator → Xstore / Xcenter	
Source	Price Sync Integrator (this system)
Target	Xstore / Xcenter (external; downstream out of scope)
Purpose	After validating and mapping the batch, the Integrator writes the DataLoader MNT file into the Xcenter inbound folder; Xstore then distributes to stores and applies at the POS.
Protocol / tech	File share (filesystem write to the Xcenter inbound folder)
Data / payload	Oracle DataLoader MNT file with FHEAD / FDETL / FDELE / FTAIL records and a matching trailer count.
Interaction & frequency	One-way file drop (one complete file written in place), once per processed nightly batch. Everything past the inbound folder is out of scope.
Security	Filesystem permissions on the shared inbound folder.

Flow 3 — Ops / Support → Price Sync Integrator	
Source	Ops / Support (the customer's operations team)
Target	Price Sync Integrator (monitoring console)
Purpose	Configure field mapping and connection settings; monitor the pipeline via dashboard, event history, and logs.
Protocol / tech	HTTPS (web console)
Data / payload	Configuration changes (write) and monitoring queries (read); the console never authors or sends prices.
Interaction & frequency	Synchronous request-response through the web UI; on-demand — configuration occasionally, monitoring as needed.
Security	Console authentication over TLS.

3.3. Containers

Internally the system is a **modular monolith**: one Spring Boot application (run as two instances for availability) that hosts the intake API, the processing worker, and the console, backed by one PostgreSQL database, behind a TLS reverse proxy (Figure 2). The reasons for one deployable rather than several services are recorded in ADR-01.

**Figure 2:** C4 level 2 - containers.

Flows. Each flow below is detailed in its own table.

Flow 1 — HQ Pricing → Application (via Reverse Proxy)	
Source	HQ Pricing (external)
Target	Price Sync Application, through the Reverse Proxy
Purpose	The nightly batch arrives; the reverse proxy terminates TLS and forwards the request to the application's intake API.
Protocol / tech	HTTPS at the edge → HTTP internally (behind the proxy)
Data / payload	Price batch (JSON) — the same envelope as the context-level flow.
Interaction & frequency	Asynchronous — intake returns HTTP 202 and the worker processes the batch later. Once per night.
Security	TLS terminated at the proxy; API key + HMAC + IP allowlist enforced at the application.

Flow 2 — Application ↔ PostgreSQL

Source	Price Sync Application
Target	PostgreSQL (Database)
Purpose	Persists events, pipeline state, audit trail, and idempotency keys; reads configuration (mapping rules, defaults, connections).
Protocol / tech	JDBC
Data / payload	Event and record rows, file metadata, audit entries, and configuration data.
Interaction & frequency	Synchronous, transactional read/write; during processing of each batch and on every configuration access.
Security	Database credentials; co-located private network with no public exposure.

Flow 3 — Application → Xcenter inbound

Source	Price Sync Application
Target	Xcenter inbound folder (file share)
Purpose	Writes the generated Oracle DataLoader MNT file to the Xcenter inbound folder.
Protocol / tech	File share (filesystem write)
Data / payload	DataLoader MNT file (FHEAD / FDETL / FDELE / FTAIL).
Interaction & frequency	One-way; the complete file is written in place, once per successfully built batch.
Security	Filesystem permissions on the inbound folder.

Flow 4 — Xcenter inbound → Oracle Xstore

Source	Xcenter inbound folder
Target	Oracle Xstore (out of scope)
Purpose	Xstore picks up the dropped file and distributes it to stores / applies at the POS.
Protocol / tech	Xstore-internal mechanism (out of scope)
Data / payload	The dropped DataLoader MNT file.
Interaction & frequency	Xstore pickup on its own schedule; out of scope for this system.
Security	Out of scope (managed by Xstore).

Flow 5 — Ops / Support → Console (via Reverse Proxy)	
Source	Ops / Support (the customer's operations team)
Target	Price Sync Application console, through the Reverse Proxy
Purpose	Ops reaches the monitoring console to view the pipeline and edit configuration.
Protocol / tech	HTTPS at the edge → HTTP internally
Data / payload	Configuration changes (write) and monitoring queries (read).
Interaction & frequency	Synchronous request-response; on-demand.
Security	Console authentication; TLS terminated at the proxy.

3.4. Daily processing flow

This subsection and the next show *how the system runs*, before the internal component structure (Figure 6). Figure 3 is one nightly run as a business process - the order of steps and where it branches. Each step's internal component interactions are realised end to end in the runtime sequence of Figure 10.

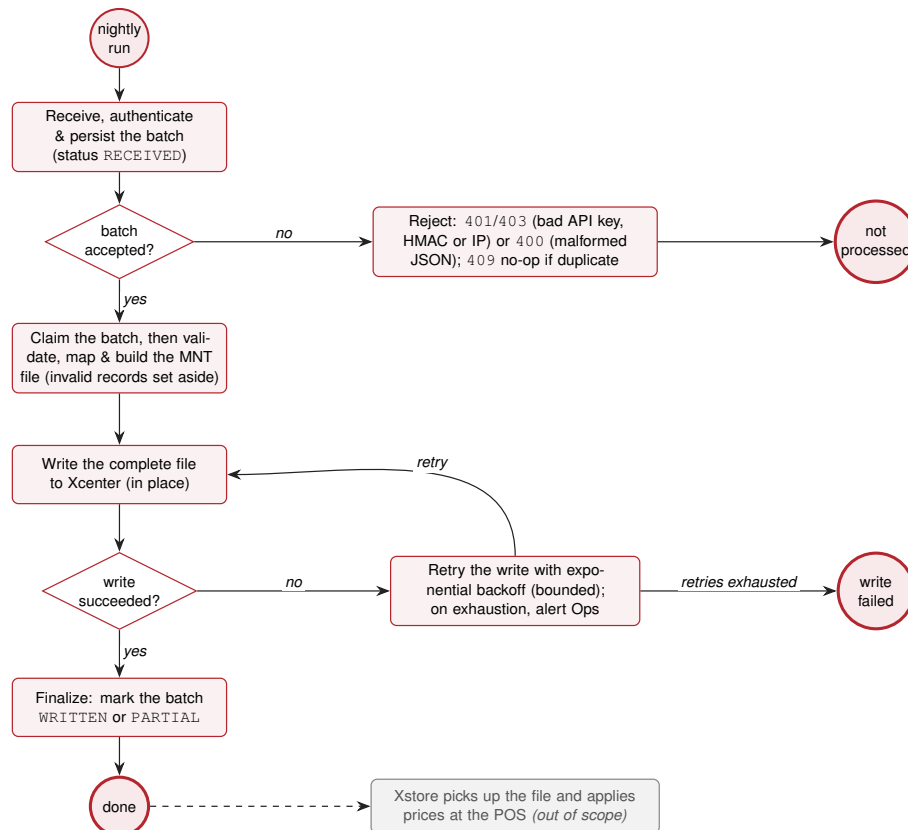


Figure 3: Daily processing flow (one nightly run).

Reading the flow. Each step maps to a design decision and a pipeline status.

Step	What happens	Status / branch
Intake	Authenticate the request (API key + HMAC + IP allowlist, FR-02), persist the raw payload and parsed records, and return 202 – processing is asynchronous (ADR-02).	RECEIVED
Decision: accepted?	Reject if authentication or format fails; acknowledge a duplicate batch with no action.	no → <i>not processed</i> (401/403/400/409); yes → continue
Claim & process	A worker claims the batch with SKIP LOCKED (ADR-04), then validates (FR-03), maps (FR-05) and builds the MNT (FR-06); invalid records are set aside (ADR-07).	PROCESSING
Write file	Write the complete file to the Xcenter inbound folder under its final name (ADR-05, FR-09).	WRITING
Decision: write ok?	On failure, retry with bounded exponential backoff (FR-12); alert Ops when retries are exhausted.	no → retry, then <i>write failed</i> ; yes → continue
Finalize	Record the outcome of the run.	WRITTEN (all valid) or PARTIAL (some set aside)
Handoff	Xstore picks up the file and applies prices at the POS.	out of scope

Table 7: Reading the daily processing flow: each step and its resulting status.

3.5. Batch lifecycle

A single batch moves through the small set of states in Figure 4; the end-to-end run (Figure 10) drives these transitions. Terminal states are drawn with a heavy border.

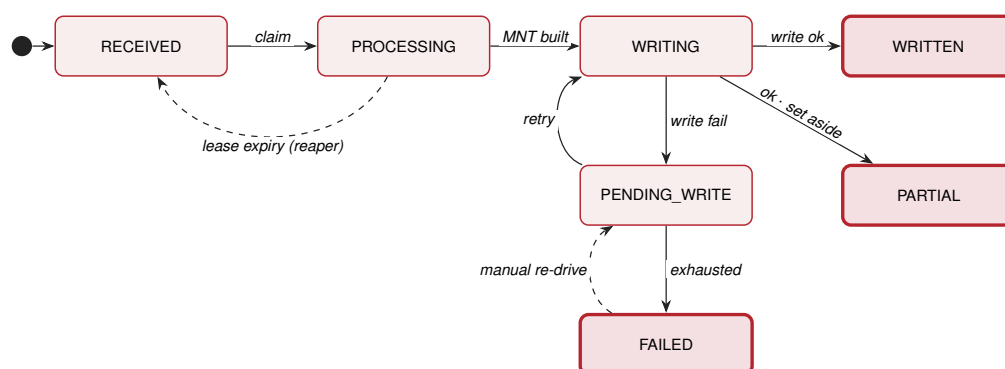


Figure 4: Batch lifecycle state machine. Solid arrows are automatic transitions; dashed arrows are recovery paths – a stale lease reclaimed by the reaper, and an operator re-drive of a FAILED batch. FAILED is terminal for automatic processing.

States. The batch holds one status at a time; terminal states (heavy border) end the run.

State	Meaning	Entered when	Leaves to
RECEIVED	Batch persisted at intake, awaiting processing.	Intake accepts and persists the batch.	PROCESSING
PROCESSING	Validate, map and build the MNT file.	A worker claims the batch (SKIP LOCKED) and takes a lease.	WRITING; or RECEIVED on lease expiry (reaper)
WRITING	Writing the file to the Xcenter inbound folder.	The MNT build is complete.	WRITTEN, PARTIAL, PENDING_WRITE; or RECEIVED on lease expiry
PENDING_WRITE	A write failed; awaiting retry.	A write attempt fails.	WRITING (retry) or FAILED (exhausted)
WRITTEN (terminal)	File written; all records valid.	Write succeeds with no set-aside records.	—
PARTIAL (terminal)	File written; some records set aside.	Write succeeds with set-aside records.	—
FAILED (terminal)	Retries exhausted; Ops alerted.	The retry limit is reached.	PENDING_WRITE (operator re-drive only)

Table 8: Batch lifecycle states and their transitions.

3.6. Architectural style and rationale

The system follows a **modular monolith with an asynchronous, database-backed work queue**. Intake persists the event and returns immediately; a worker claims and processes events from the database and performs the (network-bound) write to Xcenter.

This style fits the problem: a narrow integration with a single nightly batch, deployed on-premises, where operational simplicity and reliability matter more than independent scaling of parts. The three decisions that shape this style - one deployable (ADR-01), asynchronous processing (ADR-02), and a database queue rather than a message broker (ADR-03) - are recorded in Section 4 with the alternatives that were considered.

A fourth shaping decision keeps the style open to growth: the only destination-specific work - output format and transport - is isolated behind interfaces (ADR-09), so a new downstream system is added by plugging in implementations rather than reworking the pipeline (Section 5.7).

3.7. Technology choices

Concern	Choice and reason
Language / runtime	Java 17 (LTS) - team stack; strong Oracle Retail ecosystem fit.
Application framework	Spring Boot 3.x - web, security, data, scheduling, actuator in one stack.
Console / Web UI	Spring Model-View-Controller (MVC) REST controllers for the Console API; a full React single-page web app (built with Vite) compiled to static assets and bundled into the same deployable (ADR-01), served behind the reverse proxy - one deployable, no separate frontend host.
Persistence	Spring Data JPA / Hibernate over PostgreSQL (ADR-10) - relational state, transactions, <code>SKIP LOCKED</code> for work-claiming.
Validation	Jakarta Bean Validation + custom business rules.
Scheduling / retry	Spring Scheduling + Spring Retry (backoff).
Security	Spring Security filter chain (API key / HMAC / IP allowlist); TLS at the reverse proxy (ADR-12); secrets by reference (ADR-11).
Observability	Spring Boot Actuator + Micrometer; structured (JSON) logging.
Build / release	Maven; executable JAR / container image; Flyway database migrations.
Database	PostgreSQL 14+.
Deployment	On-premises, co-located with Xcenter; two application instances (Section 3.8).

Table 9: Technology choices and rationale.

3.8. Deployment view

Figure 5 shows where the containers of Figure 2 run. Everything is on-premises, co-located with Xcenter (the constraint of Section 1.4): two identical application instances on separate hosts (active/active, ADR-04) behind one TLS reverse proxy (ADR-12), one PostgreSQL, and the Xcenter inbound folder exported by the Xcenter host and mounted at the same path on both application hosts.

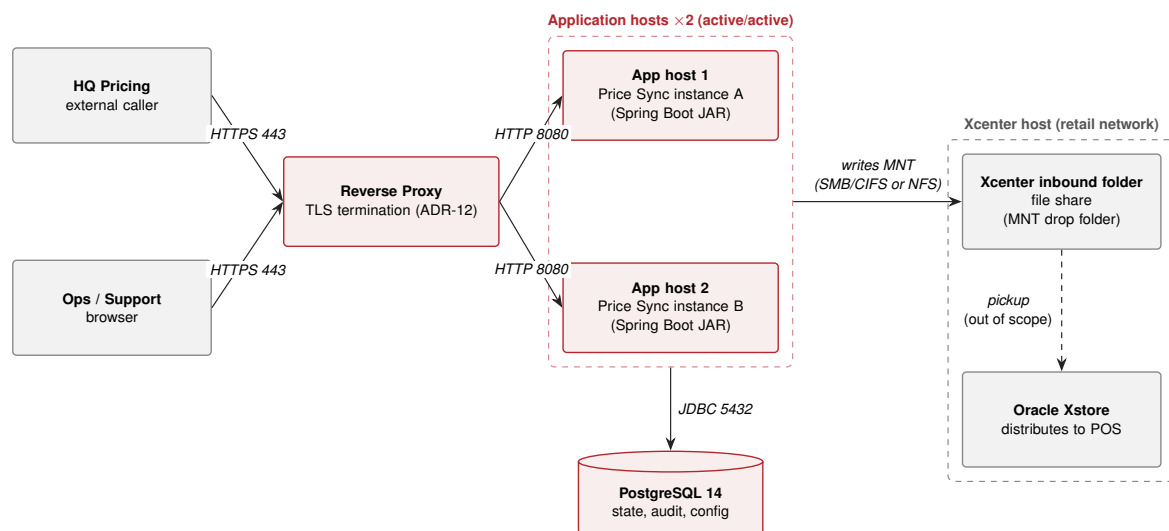


Figure 5: Deployment view – nodes, network zones, and connections.

Nodes.

Node	Runs / role	Placement notes
Reverse proxy	TLS termination, routing, edge rate limit (ADR-12).	Single published endpoint (one DNS name); health-checks the instances via /actuator/health.
Application hosts ×2	One Price Sync instance each (the Spring Boot deployable of Figure 2): intake, worker, console.	Separate hosts/VMs so a host failure leaves the nightly run intact (NFR-06); identical configuration; stateless – all durable state lives in PostgreSQL.
PostgreSQL 14	Events, records, file metadata, audit, configuration (ADR-10).	Own host/VM on the private network, no public exposure; operated and backed up under the customer's DBA practice after handover (Section 3.1).
Xcenter host	Exports the inbound folder; runs Oracle Xstore (out of scope).	Xstore ingests only complete files per its pickup contract (ADR-05, OI-2); the file is written in place, so the share need not support atomic rename.
Secret store	Resolves every <code>authRef</code> at runtime (ADR-11).	External to the application; platform keystore or vault – no secret in the database, config, or logs.
SMTP relay	Delivers alert email to the Ops group (Section 5.4).	Customer-provided relay; address held in configuration.

Table 10: Deployment nodes and placement.

Connections.

From → to	Protocol / port	Purpose and security
HQ → reverse proxy	HTTPS 443	Nightly batch; API key + HMAC + IP allowlist enforced at the application (NFR-02).
Ops → reverse proxy	HTTPS 443	Console; operator session, administrator role for writes (Section 7.3).
Proxy → instances	HTTP 8080	Plain HTTP confined to the private segment (ADR-12).
Instances → PostgreSQL	JDBC 5432	Transactional state; credentials resolved by <code>authRef</code> (ADR-11).
Instances → Xcenter share	SMB/CIFS or NFS	Service account with write access to the inbound folder only; the complete file is written in place (ADR-05).
Instances → SMTP relay	SMTP 587	Deduplicated alert email (Section 5.4).
Instances → secret store	platform-specific	<code>authRef</code> resolution at startup and on rotation (ADR-11).

Table 11: Deployment connections, ports, and security.

Operational notes. All hosts and the proxy are NTP-synchronised: the replay window on `X-Timestamp` (Section 7.5) and the `D+1` business-date default (FR-08) both assume correct clocks. The instances are stateless and interchangeable – replacing a host is redeploy-plus-mount, with no data migration – and the reverse proxy is the only published endpoint, so certificates and edge rules live in exactly one place.

4. Architecture Decision Records

Each record states the context, the options considered, the decision, and its consequences. Decisions are numbered (ADR-n) and referenced from the rest of the document.

ADR-01: Modular monolith over microservices

Status: Accepted

Context. The system is a narrow integration (receive → validate → map → write) with one nightly batch, deployed on-premises by a small team.

Options considered. (a) A single modular monolith - one deployable with clear internal modules (intake, worker, console, config). (b) Microservices - separate intake, worker, and console services. (c) Two services - engine and console split.

Decision. Option (a). One Spring Boot application with enforced module boundaries, run as two instances.

Consequences. Simplest to build, deploy, and operate on-premises; no inter-service network calls or distributed transactions. The cost is that module boundaries must be kept clean by discipline; if a part ever needs independent scaling it can be extracted later. At this volume, microservices would add infrastructure and failure modes without benefit.

ADR-02: Asynchronous processing over synchronous request handling

Status: Accepted

Context. Producing the file requires a network-bound write to Xcenter, which may be slow or temporarily unreachable; HQ should not be blocked or made to handle that.

Options considered. (a) Synchronous - validate, map, build, and write inside the HTTP request, returning success only after the file is written. (b) Asynchronous - persist the event on intake, return 202 Accepted, and process and write in a background worker.

Decision. Option (b). Intake persists at status RECEIVED and returns 202; a worker validates, maps, builds, and writes.

Consequences. Intake stays fast and available regardless of Xcenter; failed writes can be retried without losing the event (FR-12). The cost is eventual (not immediate) completion and the need to track pipeline state and poll for work - both already required by FR-11 and the audit trail.

ADR-03: Database-backed work queue over a message broker

Status: Accepted

Context. Asynchronous processing needs a place to hold work between intake and the worker; the system already needs PostgreSQL for state, audit, idempotency, and configuration.

Options considered. (a) A PostgreSQL table used as the work queue (poll + claim). (b) A message broker (Kafka / RabbitMQ). (c) An in-memory queue inside the application.

Decision. Option (a). Events live in the database; the worker claims unprocessed rows.

Consequences. No extra infrastructure to deploy and operate on-premises; work and state share one transactional store, so claiming and status updates are consistent. A broker is unjustified at one batch per night; an in-memory queue would lose events on a crash. The cost is short polling latency, which is irrelevant for a nightly batch.

ADR-04: Work-claiming with SELECT ... FOR UPDATE SKIP LOCKED across two instances

Status: Accepted

Context. Two application instances run for availability. Both could try to process the same event, which must never cause a double write.

Options considered. (a) `SELECT ... FOR UPDATE SKIP LOCKED` - each instance atomically claims unclaimed events and skips rows another instance holds. (b) Leader election / a scheduler lock (e.g. ShedLock) so only one instance processes at a time. (c) Single active instance with a passive standby. (d) A PostgreSQL advisory lock around processing.

Decision. Option (a), refined for the asynchronous pipeline. *Claiming is a short transaction:* the worker runs `SELECT ... FOR UPDATE SKIP LOCKED` over `RECEIVED` rows, flips the row to `PROCESSING`, stamps an ownership lease (owning-instance id + a `claimed_at` heartbeat), and commits – so the row lock is released at once and is never held across the network-bound write.

Validation, mapping, build and the Xcenter write then run *outside* that transaction, each committing its own status change (`WRITING`, then `WRITTEN/PARTIAL` or `PENDING_WRITE`). A periodic reaper sweep resets any `PROCESSING/WRITING` row whose lease has expired (its owner crashed mid-run) back to `RECEIVED` so it can be re-claimed.

Consequences. Both instances stay active (true high availability, HA) with no double-processing: `SKIP LOCKED` stops two workers claiming the same row, and the reaper releases a row only after its lease has *expired*, so a slow-but-alive worker is never pre-empted.

No long-running transaction is held during the file write, which avoids connection-pool exhaustion and table bloat. The cost is the two lease columns plus a reaper sweep, and a lease timeout that must sit safely above the longest expected processing time. It relies on PostgreSQL row-locking semantics, which are well defined. Leader election adds a coordination concept; an active/standby pair wastes a node and needs failover detection.

ADR-05: Write the MNT file directly to the Xcenter inbound folder

Status: Accepted

Context. Xstore must never *ingest* a half-written file from the Xcenter inbound folder. How Xstore picks files up therefore decides how the write must be done.

Options considered. (a) Write to a temporary name, flush, then atomically rename to the final name. (b) Write the final file and signal completion with a separate done-flag / sentinel file. (c) Write the complete file directly under its final name, relying on Xstore's pickup contract not to ingest an incomplete file.

Decision. Option (c). The Xcenter Writer assembles the file locally (streamed to a local temporary file for bounded memory) and, once complete, writes it directly to the Xcenter inbound folder under its final `.mnt` name. This is safe because the Xstore pickup contract does not ingest an incomplete file – pickup is gated (an explicit trigger, or a completeness/stability check) rather than grabbing any `.mnt` the instant it appears (OI-2, confirmed with the integration owner). Writing in place keeps the writer simple: no destination temporary file, no rename step, and no requirement that the share support atomic rename.

Consequences. The write path is a single step and no longer depends on same-volume atomic-rename semantics, which a network share (SMB/CIFS or NFS) may not guarantee – this is the simplification the decision buys. The trade-off is that safety now rests entirely on the Xstore pickup contract (OI-2): a failed or interrupted write can leave an incomplete file in the folder, and the design relies on Xstore not ingesting it; the Retry Scheduler then re-drives the batch and rewrites the file until it succeeds. If a deployment's Xstore cannot guarantee this pickup contract (e.g. it auto-scans and ingests immediately), the safer temp-then-rename or sentinel-flag modes can be reintroduced behind the same `OutputWriter` interface (ADR-09) without touching the pipeline.

ADR-06: Two-level idempotency: batch identity and record identity

Status: Accepted

Context. A nightly batch may be re-sent (e.g. after a timeout), and an individual price change may be superseded by a newer version; neither must create duplicate or stale output (FR-13).

Options considered. (a) Two levels - a unique batch key (batch id + version) to drop a re-sent batch, plus a record key (change id + version) so the latest version of a change wins. (b) A single request-id dedupe at the batch level only. (c) No idempotency, relying on HQ not to resend.

Decision. Option (a). Unique constraints at both levels; a superseding record version replaces the older one.

Consequences. Safe under at-least-once delivery and corrections; no duplicate files and no stale prices. The cost is the upsert / supersede logic and the unique constraints. Batch-only dedupe would miss record-level supersede; no idempotency risks incorrect files.

ADR-07: Partial handling: set aside invalid records and continue

Status: Accepted

Context. A single bad or unmappable record should not block an otherwise valid nightly batch of tens of thousands of prices.

Options considered. (a) Set-aside-and-continue - write the valid records, flag the invalid ones, mark the event `PARTIAL` (FR-03, FR-05). (b) All-or-nothing - reject the whole batch if any record is invalid.

Decision. Option (a) as the default. A configurable threshold escalates to abort when too many records are bad – default: abort if more than 20% of records are set aside, or if no valid record remains; otherwise the run finishes `PARTIAL`.

Consequences. One bad stock-keeping unit (SKU) does not delay all good prices; set-aside records are reported for follow-up and the trailer counts only written rows. The cost is the need to track and surface set-aside records. All-or-nothing is safer for strict consistency but blocks the whole run, so it is offered only above the threshold.

ADR-08: Configuration in the database over config files or hard-coding

Status: Accepted

Context. Mapping rules, value maps, defaults, scope, and connection endpoints change as stores/regions are added; the URD requires this without core-code change (NFR-09, FR-15).

Options considered. (a) Hold configuration in the database, edited through the console. (b) Hold it in config files deployed with the application. (c) Hard-code it.

Decision. Option (a). A Config Service reads configuration from the database; the console edits it; changes are audited.

Consequences. New stores/regions and mapping changes need no redeploy or restart, and changes are auditable. The cost is a config service and console create / read / update / delete (CRUD) plus a cache/reload. Config files would need a redeploy or restart; hard-coding violates the extensibility requirement.

ADR-09: Pluggable output targets behind format and transport interfaces

Status: Accepted

Context. Today the only destination is Oracle Xstore via a DataLoader MNT file dropped on a share. HQ may later need the same price feed delivered to a *different* downstream system, in a different format (e.g. XML or a REST payload) and over a different transport (e.g. SFTP or an HTTP API). The design must let a new destination be added without reworking intake, validation, or orchestration (NFR-09).

Options considered. (a) Abstract the two destination-specific concerns - output *format* and output *transport* - behind interfaces (`PayloadBuilder`, `OutputWriter`), pick the implementation by configuration, and keep the pipeline on a target-neutral internal record. (b) Branch inside one builder/writer with per-destination conditionals. (c) Fork the service per destination.

Decision. Option (a). Records are validated into a target-neutral form; a per-target `PayloadBuilder` renders the artifact and a per-target `OutputWriter` delivers it, both resolved from configuration (mapping profile + connection). The current Xstore path is one implementation pair (`XstoreMntBuilder` + `XcenterWriter`).

Consequences. Adding a destination is *additive*: implement the two interfaces and add a mapping / connection profile in configuration - intake, security, validation, dispatch, persistence, idempotency, retry, audit, and the console stay untouched (Open/Closed). The cost is the two abstractions plus a small target registry. Conditionals grow fragile as targets multiply; forking the service duplicates the whole pipeline. The workflow is detailed in Section 5.7.

ADR-10: PostgreSQL as the system database (not Oracle DB)

Status: Accepted

Context. The customer runs an Oracle Retail ecosystem and likely holds Oracle DB licences, so using Oracle DB for this system's own state is an obvious alternative.

Options considered. (a) PostgreSQL as the application's own state store. (b) Oracle DB, reusing the existing licence/estate.

Decision. Option (a). The system owns a small, self-contained schema (events, records, file metadata, audit, config) and depends only on portable SQL plus `FOR UPDATE SKIP LOCKED`, which PostgreSQL supports natively.

Consequences. No coupling to the customer's Oracle estate or its DBA/change processes for this integrator's data; lower licence and footprint for an on-prem app; the team's stack fit (ADR refers to Section 3.7). `SKIP LOCKED` and `jsonb` (raw-payload storage) are first-class. The cost is one more engine to run on-prem; if the customer mandates Oracle DB, the persistence is standard enough to port, since no PostgreSQL-only feature beyond `SKIP LOCKED`/`jsonb` is relied on (Oracle has equivalents).

ADR-11: Secret management: no plaintext secrets

Status: Accepted

Context. The system handles secrets - the HQ HMAC shared secret and API key, the database credentials, and the Xcenter share credentials - and NFR-02 forbids storing any secret in plaintext.

Options considered. (a) An external secret store / OS keystore (e.g. Vault, a mounted secret, or the platform credential store), with the database holding only a reference (`authRef`). (b) Encrypted-at-rest columns in the database with a key kept outside it (e.g. Jasypt). (c) Secrets in config files or environment as plaintext.

Decision. Options (a)/(b): `ConnectionConfig.authRef` is a *reference* (key name / path) resolved at runtime from the secret store; secrets are never written to the database, logs, audit entries, or the console. Where a value must live in the database it is stored encrypted with an externally held key. Secrets are rotatable without redeploy by updating the store and the reference.

Consequences. NFR-02 is met; a database or backup leak exposes no usable secret; rotation is operational, not a code change. The cost is a dependency on a secret store and the discipline to keep secrets out of logs. Option (c) is rejected outright.

ADR-12: TLS termination at a reverse proxy

Status: Accepted

Context. Both callers (HQ machine-to-machine intake, and Ops on the console) connect over HTTPS; the application itself should stay focused on business logic, not TLS plumbing.

Options considered. (a) Terminate TLS at a reverse proxy in front of the app, forwarding HTTP on a co-located private network. (b) Terminate TLS inside each application instance. (c) Mutual TLS end-to-end.

Decision. Option (a). The proxy terminates TLS and routes intake and console traffic to the app; application-level authentication (API key + HMAC + IP allowlist) is still enforced *at the app*, so the proxy is not a trust boundary on its own.

Observability is via Spring Boot Actuator + Micrometer with structured JSON logs (Section 3.7), chosen as the stack-native, no-extra-infrastructure option consistent with ADR-01/ADR-03.

Consequences. Certificate handling, routing, and rate-limit/edge concerns live in one place; the two app instances sit behind one endpoint for the load balancer. The internal hop is plain HTTP but confined to the co-located private network. The cost is one more component to operate; mTLS (c) is available if HQ later requires it.

5. Component Design (C4 level 3)

5.1. Component overview

This section zooms into the **Price Sync Application** container from Figure 2 and shows the components inside it (Figure 6).

They fall into three groups: *intake* components that accept and persist an event, *processing* components that claim an event and run the validate-map-build-write pipeline, and *supporting* components (scheduling, configuration, console, alerting). PostgreSQL and the Xcenter inbound folder are neighbouring elements, shown outside the application boundary.

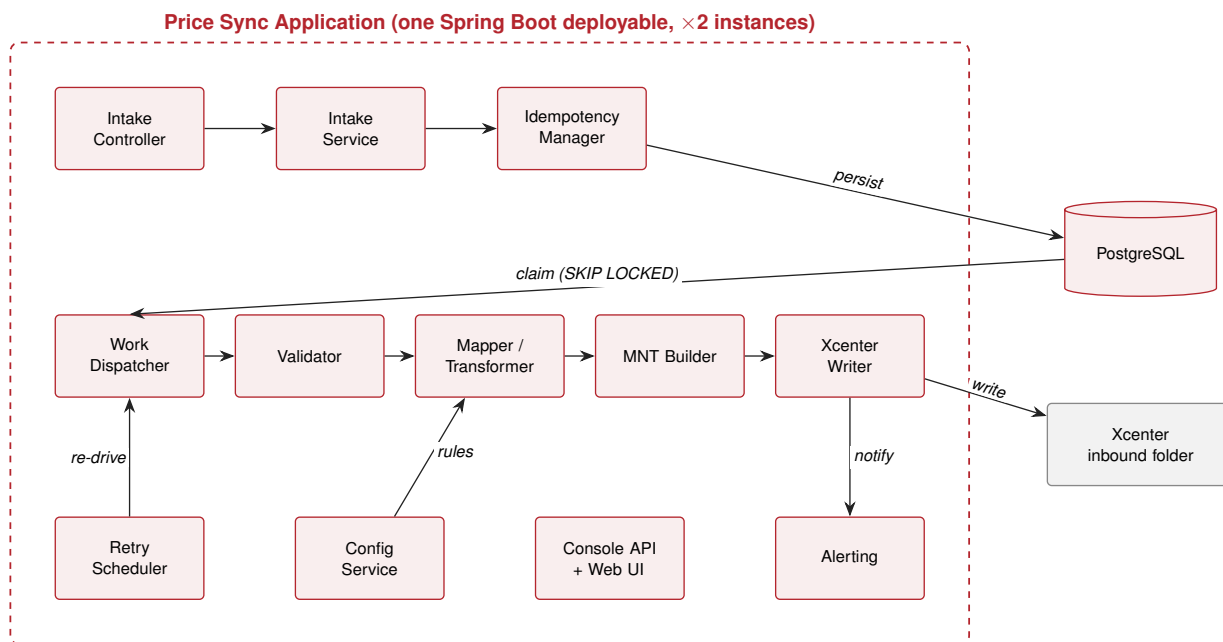


Figure 6: C4 level 3 - components inside the application.

Note: the arrows show only the batch-processing pipeline. The supporting components – **Console API + Web UI**, **Config Service**, and **Retry Scheduler** – all read and write PostgreSQL through Spring Data repositories as well; the Console, for instance, serves the dashboard, event list/detail, logs, and configuration CRUD entirely from the database (FR-15, FR-16). These data-access arrows would touch nearly every box, so they are omitted here for clarity (the same convention as Figure 8). A component drawn without an outgoing arrow is therefore reached over one of these implied repository links, not disconnected.

5.2. Intake components

Intake Controller. The REST entry point for `POST /api/v1/price-events`. It enforces the security filter chain (API key / HMAC / IP allowlist), binds and lightly validates the request body, delegates to the Intake Service, and returns `202 Accepted` with the batch id. It performs no mapping or writing. *Failure:* returns `4xx` for auth/format errors. *Concurrency:* stateless; safe on both instances.

Intake Service. Encapsulates intake logic: it asks the Idempotency Manager whether the batch is new, persists the raw payload (`jsonb`) and the parsed `PriceBatch/PriceRecord` rows at status `RECEIVED`, and writes the first audit entry. Keeping logic here keeps the controller thin and

testable.

Idempotency Manager. Enforces the two-level identity from ADR-06: it rejects a re-sent batch (same `batch_id+version`) and, per record, lets a higher `change_id+version` supersede an older one. Backed by unique constraints, so concurrent duplicates fail safely at the database.

5.3. Processing components

Work Dispatcher. A scheduled poller (polls every ~10s) that claims `RECEIVED` events with `SELECT ... FOR UPDATE SKIP LOCKED` in a *short* transaction – it flips the row to `PROCESSING`, stamps an ownership lease (owner instance + `claimed_at` heartbeat, renewed every minute, 5 min timeout) and commits, releasing the lock at once (ADR-04) – then runs `validate / map / build / write` *outside* that transaction, committing each status change as it goes. *Concurrency:* `SKIP LOCKED` guarantees only one instance claims a given event (no double write); if an owner crashes mid-run, the reaper resets its expired lease back to `RECEIVED` for re-claim (S5). Both instances stay active.

Validator. Applies the intake business rules per record (FR-03): required fields, `price > 0`, valid currency, `effective_start` before `effective_end`.

"Known item / location" is checked against the reference data the system holds in configuration – the item and store/zone codes in the mapping profile (ADR-08), refreshed from Xstore by a daily sync that runs before the nightly batch; the system does *not* own the Xstore catalog. A record whose code is unknown is therefore an *unmappable* record (FR-05), not a hard format error.

A failing record is marked invalid with a reason and set aside; the batch continues (ADR-07).

Mapper / Transformer. Converts each valid record into a `MntRow` using configurable rules from the Config Service (FR-05): field-to-column mapping, value maps (`change_type` `new/update`→`FDETL`, `delete`→`FDELE`), and defaults (`CURRENCY` – deployment default, e.g. `VND`, configurable; `EFF_START_DATE`=`D+1` when absent, FR-08).

`LOC_TYPE` is *derived per record* from whether `store_id_or_zone` resolves to a single store or a zone/region (default codes `S` for store, `Z` for zone, held in the value map and configurable per the Host Interface Guide) – it is not a constant, so a zone record is never mislabelled as a store. An unmappable record is set aside and flagged.

MNT Builder (XstoreMntBuilder). The Xstore implementation of the `PayloadBuilder` interface: it assembles the `DataLoader` file from the mapped rows (FR-06) - a `FHEAD` line, one `FDETL`/`FDELE` line per record (fixed column order per the Oracle Host Interface Guide; default comma delimiter, RFC-4180-style quoting for fields containing a delimiter/quote/newline, UTF-8 – all configurable), and a `FTAIL` line whose count equals the total detail rows written, counting both `FDETL` and `FDELE`.

Rows are *streamed* to the temporary file as they are mapped rather than buffered whole in memory, so memory stays bounded regardless of catalog size (NFR-01). A different target supplies its own `PayloadBuilder` (ADR-09).

Xcenter Writer (XcenterWriter). The Xstore implementation of the `OutputWriter` interface: it writes the assembled file directly to the Xcenter inbound folder under its final name (ADR-05, FR-09); Xstore ingests only complete files per its pickup contract, so no destination temp file or rename step is needed.

The final filename follows the Host Interface Guide naming convention and is made unique per produced file – default pattern `pricesync_<batch_id>_v<version>_<timestamp>.mnt` – so a same-night superseding version (v2) lands as a distinct file rather than colliding with or

overwriting the earlier one.

On failure it marks the event `PENDING_WRITE` for the Retry Scheduler and raises an alert path. A different target supplies its own `OutputWriter` (ADR-09). *Concurrency*: each file has a unique per-event name (`batch_id+version+timestamp`), so two writers never target the same file.

5.4. Supporting components

Retry Scheduler. A scheduled job (polls every ~10 s) that *claims* `PENDING_WRITE` events with `SELECT ... FOR UPDATE SKIP LOCKED` – so the two instances never re-drive the same batch – and re-drives each through the writer with exponential backoff (30 s, doubling, capped at 10 min) up to 6 attempts (~30 min total; FR-12, NFR-04); on exhaustion it marks the event `FAILED` and alerts.

A `FAILED` batch is *not* retried automatically again: an operator re-drives it from the console (Section 7.3, `POST /events/{id}/retry`), which returns it to `PENDING_WRITE` for this job to pick up.

Config Service. Reads mapping rules, value maps, defaults, scope, output format (delimiter, quoting), and connection settings from the database (ADR-08); serves them to the Mapper and the MNT Builder, and exposes them to the console for editing.

Each cached entry carries the row's `updated_at/version`; the cache uses a short time-to-live (TTL) and is *reloaded at the start of each batch*, and each instance checks the version so an edit made on one instance is picked up by the other within the TTL. Because configuration changes are occasional and processing is a nightly batch, this bounded staleness window is acceptable (no edit is ever applied mid-batch).

Console API + Web UI. Read-mostly endpoints backing the Ops console: dashboard metrics, event list and detail (including set-aside records), logs, and configuration CRUD (FR-15, FR-16).

Implementation. The Console API is a set of Spring MVC REST controllers hosted *inside* the same Spring Boot application (no separate service – ADR-01), reading through the same Spring Data repositories as the pipeline.

The Web UI is a full **React** single-page web app (built with Vite) that consumes the Console REST API; it is compiled to static assets and bundled into the one deployable, served behind the reverse proxy, so there is no separate frontend host to operate – still one deployable (ADR-01), while giving Ops a richer, interactive console (live dashboard, searchable event and log views, configuration forms).

Operators authenticate as users; configuration writes are restricted to an administrator role. Monitoring only - it never authors or sends prices.

Alerting. Raises alerts on two channels (FR-17, FR-18): the console **dashboard** and **email** to an Ops distribution group (default e.g. `ops-alerts@customer.com`; the address is held in configuration and editable in the console, not hard-coded – ADR-08).

All three critical conditions notify: *no batch received before the nightly cut-off* – a missing-batch watchdog, since the system is the *receiver* and cannot observe HQ directly; repeated Xcenter write failures; and a batch reaching `FAILED` (retries exhausted).

Alerts are deduplicated so each incident raises *one* email rather than a flood (e.g. a multi-hour Xcenter outage is a single notification, not one per retry).

Persistence / Repositories. Spring Data repositories over PostgreSQL provide all components with transactional access to events, records, files, audit entries, and configuration; the application holds no other durable state, which is what allows two stateless instances.

Retention / Purge. A scheduled job deletes terminal batches (WRITTEN/PARTIAL/FAILED) and their records, file metadata, and audit entries once they are older than the configured retention window (≥ 2 months, NFR-08). Because purging also removes the idempotency keys of those batches, the window must stay comfortably longer than any plausible HQ re-send delay so a legitimate re-send is never re-processed as new (Appendix B).

5.5. Static structure - domain model

Figure 7 shows the main domain entities and value objects and how they relate:

- **PriceBatch** - one nightly batch (batch_id, version, status); parent of many PriceRecords.
- **PriceRecord** - one item/location price change parsed from the HQ payload.
- **MntRow** - value object derived from a valid PriceRecord; one DataLoader detail line.
- **MntFile** - the assembled DataLoader file built from MntRows.
- **MappingRule**, **ConnectionConfig** - configuration entities (field/value maps and defaults; connection settings).
- **AuditEvent** - one audit entry recorded per pipeline step.

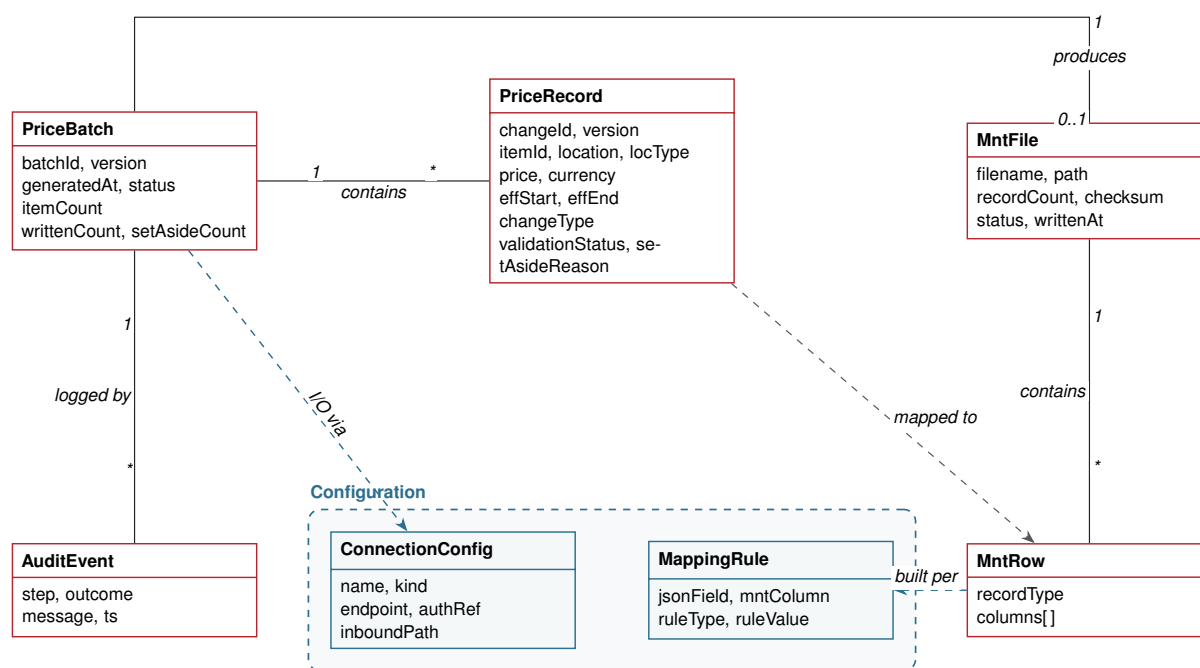


Figure 7: Domain model - entities and value objects.

Two kinds of box. The red entities are the data that flows through the pipeline - a batch, its records, the file it produces, the rows in that file, and the audit trail. The two slate **configuration** entities sit apart because they are not part of that data: they are reference settings read while a batch is processed.

How configuration is used. Each **MntRow** is built according to a **MappingRule** (which JSON field becomes which MNT column), and both the batch intake and the Xcenter write use a **ConnectionConfig** (endpoint, credentials, inbound path).

Arrows. Solid lines are associations - a stored reference, for example a record that knows its batch. Dashed lines are derivation / configuration-use dependencies: the link exists only during processing and is not persisted, for example a `MntRow` is derived from a `PriceRecord` but nothing stores that link.

5.6. Static structure - key application classes

Figure 8 shows the principal application classes and their collaborations. The controller delegates to the Intake Service; the Work Dispatcher orchestrates the pipeline classes; and the destination-specific work sits behind two interfaces, so a new target is added by implementing them rather than by changing the pipeline (Section 5.7).

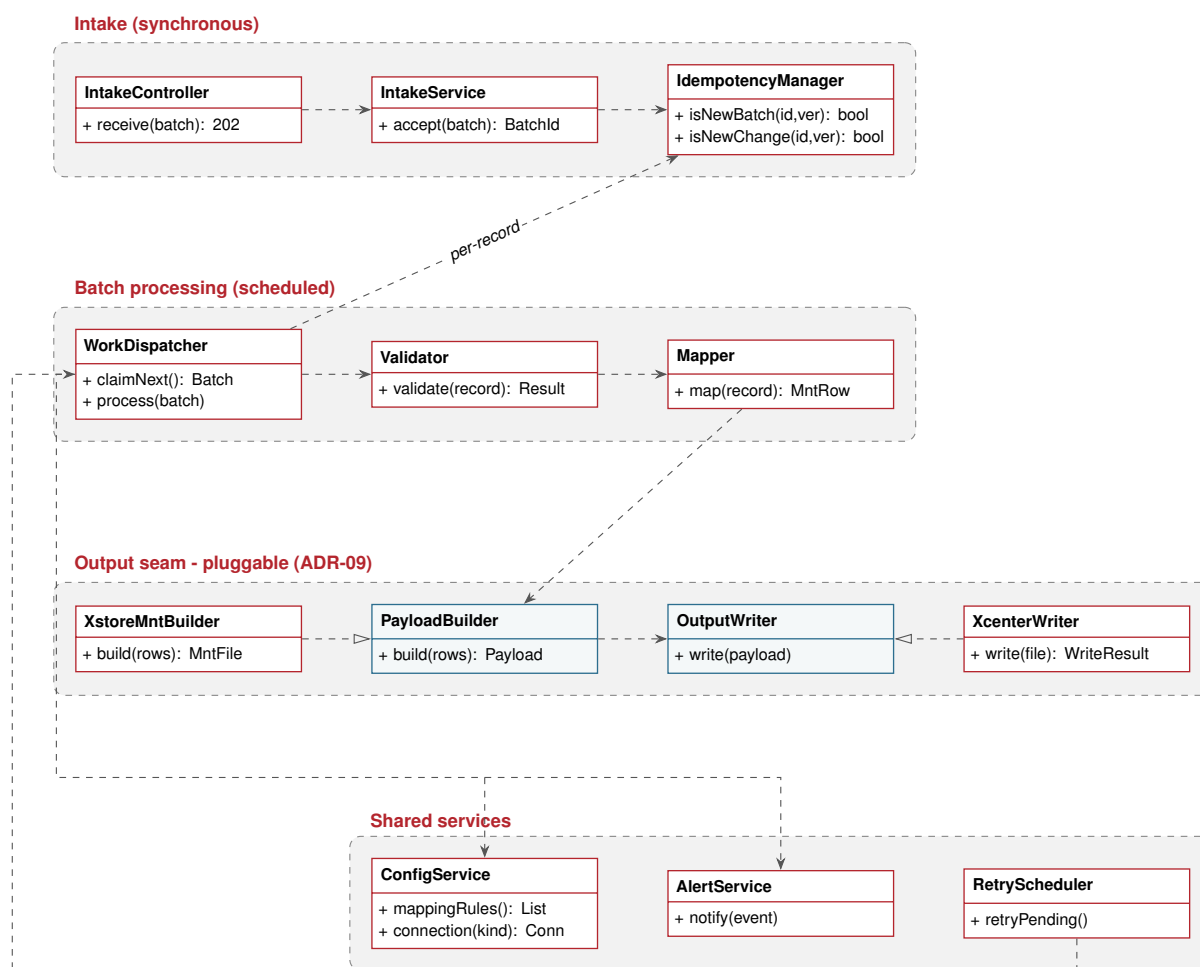


Figure 8: Key application classes grouped by responsibility, with their collaborations and the pluggable output seam.

Four bands. The classes are grouped by responsibility: synchronous intake, scheduled batch processing, the pluggable output seam, and shared services.

The output seam. The slate boxes are the two output-port interfaces, `PayloadBuilder` (format) and `OutputWriter` (transport). `XstoreMntBuilder` and `XcenterWriter` are the adapters that realise them (hollow-triangle arrows). A new target needs only new adapters, leaving the pipeline untouched (ADR-09, Figure 9).

Flow and idempotency. Dashed arrows show how the classes call or hand off to one another; the main processing flow is `validate` → `map` → `build` → `write`. Idempotency is checked at intake (batch

level) and during processing (per record).

Shared services. `WorkDispatcher` loads configuration from `ConfigService` and is the single place raising alerts via `AlertService` (set-aside records, partial batches, write failures). `RetryScheduler` re-drives it for any batch left in `PENDING_WRITE` - the loop on the left. Repositories are omitted for clarity.

5.7. Extensibility – adding a new output target

The pipeline up to and including validation is *target-neutral*: it produces canonical, validated `PriceRecords`. Only the final two steps are destination-specific, and both sit behind interfaces (ADR-09): `PayloadBuilder` renders the records into a target's artifact, and `OutputWriter` delivers that artifact over the target's transport.

The Work Dispatcher resolves the pair from configuration (the target's mapping profile and connection), so the core never names a concrete destination. Figure 9 shows the current Xstore pair and how a future target slots in beside it.

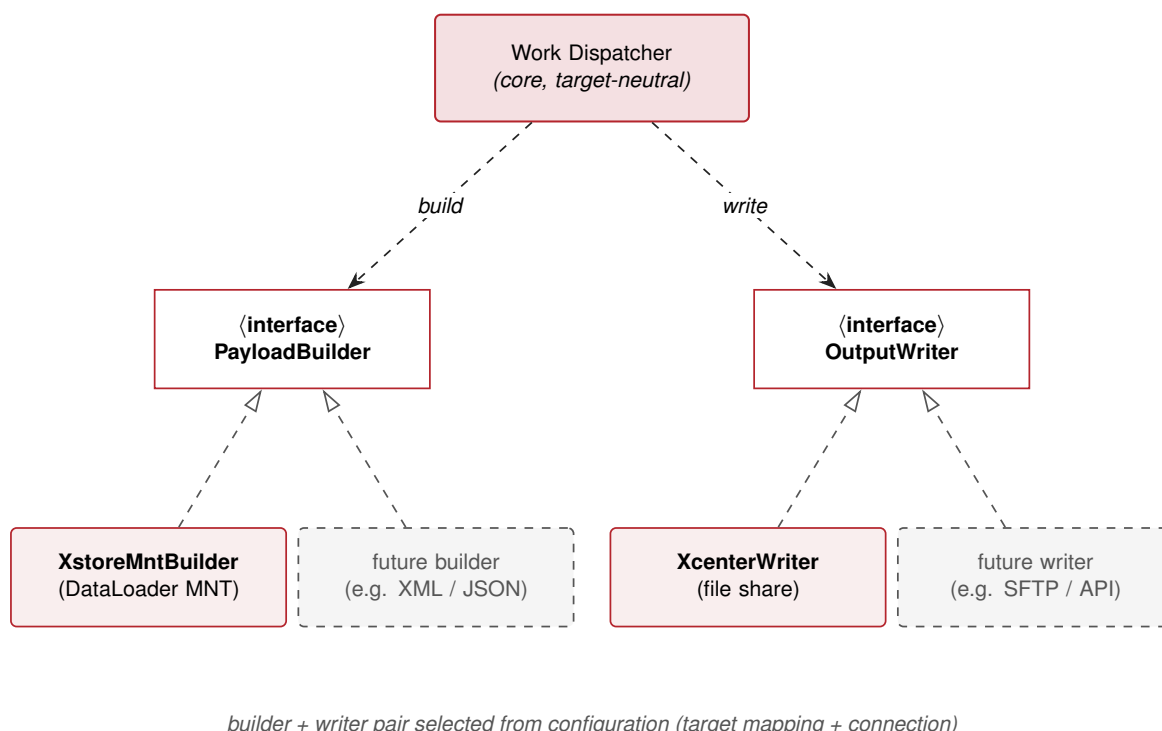


Figure 9: Output-target extension seam.

To add a destination (for example a partner system), the work is purely additive:

1. Implement `PayloadBuilder` for the target's format (e.g. XML, JSON, a fixed-width file).
2. Implement `OutputWriter` for the target's transport (e.g. SFTP, a REST API, a message queue).
3. Add the target's mapping profile, defaults, and connection in configuration (ADR-08) - no code for mapping.
4. Register / enable the target so the dispatcher can route to it (one target, several targets, or a different target per route).

Nothing in intake, security, validation, dispatch, persistence, idempotency, retry, audit, or the console changes - this is the Open/Closed property the design is built for (NFR-09).

6. Runtime Scenarios

This section is the dynamic view of the design. It opens with the **main success flow** – one normal nightly run, end to end – and then details each point where that flow can branch as a **component-level use-case realisation**: an *alternative flow* (a valid variation that still completes) or an *exception flow* (an error condition and the recovery the system performs), each a sequence across the components of Figure 6 that ends in one of the batch states of Figure 4.

The emphasis is on *what the system does automatically* – claim, validate, set aside, retry, fail over, alert – with no operator intervention.

6.1. Runtime sequence – end-to-end daily sync [main success path with branch points]

Purpose. The dynamic counterpart of the component view in Figure 6: one realisation of the normal nightly run, end to end, drawn at the *subsystem* level (Intake, Batch Processor, Database, Output Writer) so the whole flow fits a single view.

The happy path is the [else]/[valid]/[write ok] branch of each fragment; the fragments show where the run can diverge, and the table below summarises each branch with a pointer to its component-level realisation (S1–S5) below. Every path ends in a terminal state of Figure 4.

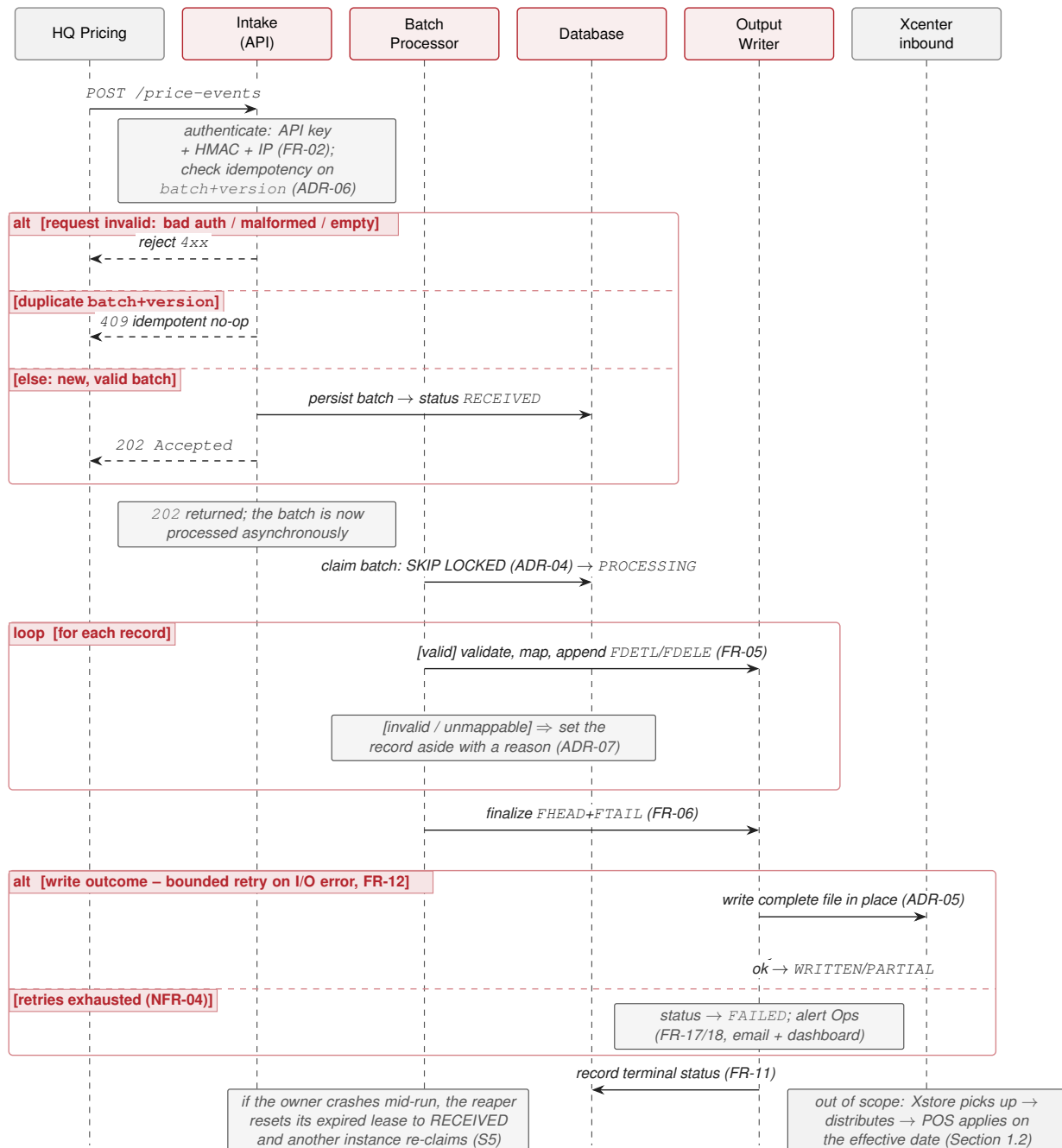


Figure 10: End-to-end daily sync: the main success path (the [else]/[valid]/[write ok] branches) with the intake, per-record, and write fragments where the run can diverge.

Main success path.

1. HQ posts the nightly batch to the intake endpoint over TLS, with an API key, an HMAC-SHA256

body signature, and from an allowlisted IP (FR-01, FR-02, NFR-02).

2. Intake authenticates, parses the envelope, and checks idempotency on `batch_id+version` (ADR-06). For a new, well-formed batch it persists the batch as `RECEIVED` and returns `202 Accepted`; HQ's responsibility ends here – processing is asynchronous.
3. A processing instance claims the `RECEIVED` batch with `SELECT ... FOR UPDATE SKIP LOCKED` (ADR-04) and moves it to `PROCESSING`; `SKIP LOCKED` guarantees exactly one of the two instances claims it.
4. For each record the Batch Processor validates the business rules (price > 0, valid currency, start before end, valid item/store group – FR-03), maps HQ item/location codes to Xstore codes and applies date defaults (FR-05, FR-08), and appends an `FDETL` (create/modify) or `FDELE` (delete) row.
5. After the loop it finalises the file – `FHEAD` header and `FTAIL` trailer whose record count equals the detail rows written (FR-06) – so the file is self-checking.
6. The Output Writer writes the complete file to the Xcenter inbound folder under its final name (ADR-05, FR-09); Xstore ingests only complete files per its pickup contract, so a partial write is never picked up downstream.
7. On success the batch finalises as `WRITTEN` (or `PARTIAL` if any records were set aside) and the terminal status is recorded (FR-11).
8. Downstream is out of scope (Section 1.2): Xstore picks up the file, distributes it to stores, and each POS applies on the effective date.

Where the run branches. Each fragment in Figure 10 is a branch the system handles automatically; the *Detail* column points to the component-level scenario (S1–S5) that realises it. All converge on a terminal state of Figure 4.

Branch point	Condition	What the system does (no operator action)	Terminal	Detail
Intake	Bad auth / malformed / empty batch	Reject with <code>4xx</code> at the edge; nothing is persisted.	none	S1, Fig. 11
Intake	Same <code>batch_id+version</code> re-sent	409; idempotent no-op, original batch unchanged (ADR-06).	unchanged	S2, Fig. 12
Per record	Record invalid or unmappable	Set the record aside with a reason (ADR-07); write the rest.	<code>PARTIAL</code>	S3, Fig. 13
Write	Xcenter I/O error (share down, disk, perms)	Hold at <code>PENDING_WRITE</code> , retry with bounded backoff (FR-12, NFR-04); alert only if exhausted.	<code>WRITTEN/PARTIAL</code> or <code>FAILED</code>	S4, Fig. 14
Mid-run	Processing instance crashes	The owner's lease expires; the reaper resets the batch to <code>RECEIVED</code> and the other instance re-claims via <code>SKIP LOCKED</code> (ADR-04); no double write.	resumes → <code>WRITTEN</code>	S5, Fig. 15

Table 12: End-to-end daily sync: divergence points and their outcomes.

6.2. S1 – Intake rejection [exception flow]

Trigger. HQ posts a batch that fails authentication or basic format checks. The request is rejected at the edge by the Intake Controller's security filter chain before anything is persisted, so no batch row is ever created.

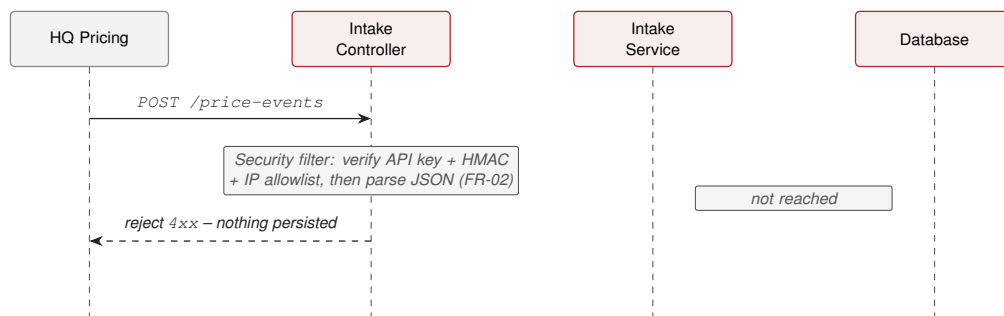


Figure 11: S1 – intake rejection (no persistence).

Exception handling. The condition determines the status code; every rejection is final (the caller fixes the request and resends – these are *not* retryable).

Condition	HTTP	error	Caller action
Bad / missing API key or HMAC	401	UNAUTHENTICATED	Check key, signature, and clock; resend.
Source IP not allowlisted	403	IP_NOT_ALLOWED	Send from an allowlisted address.
Malformed JSON / missing envelope fields	400	MALFORMED_REQUEST	Fix the payload and resend.
Empty <code>records</code> array	422	EMPTY_BATCH	Resend with at least one record.

Table 13: S1 intake rejection: conditions, HTTP status, and caller action.

6.3. S2 – Duplicate batch [alternative flow]

Trigger. HQ re-sends a batch it already sent (e.g. after a network timeout where the first 202 was lost). The batch authenticates correctly, but the Idempotency Manager recognises the `batch_id+version` and the system performs an idempotent no-op (ADR-06) instead of producing a second file.

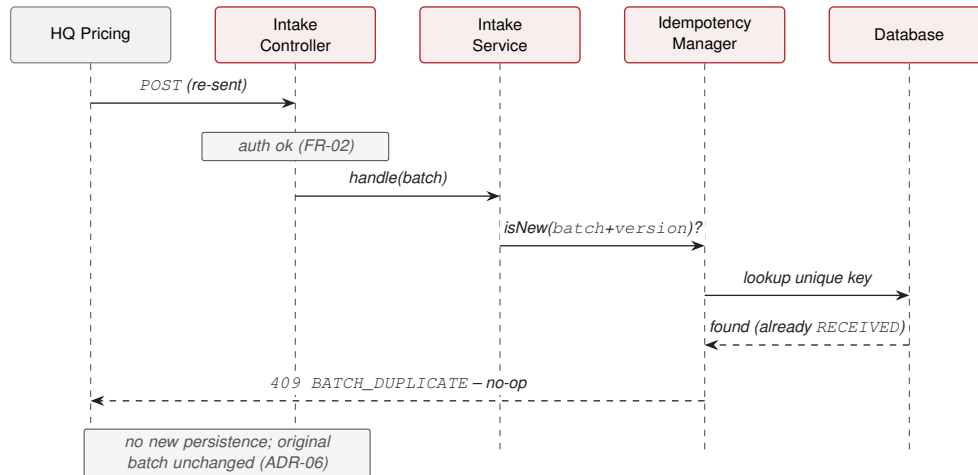


Figure 12: S2 – duplicate batch (idempotent no-op).

Outcome. No state change; the original batch keeps its status. Record-level identity works the same way during processing: a higher `change_id+version` supersedes an older one, a lower or equal one is ignored (ADR-06).

6.4. S3 – Partial batch [alternative flow]

Trigger. A claimed batch contains records that fail validation (FR-03) or cannot be mapped (FR-05). Those records are set aside with a reason (ADR-07); the rest are written normally, so the file is produced and the batch finalises as `PARTIAL` rather than failing wholesale.

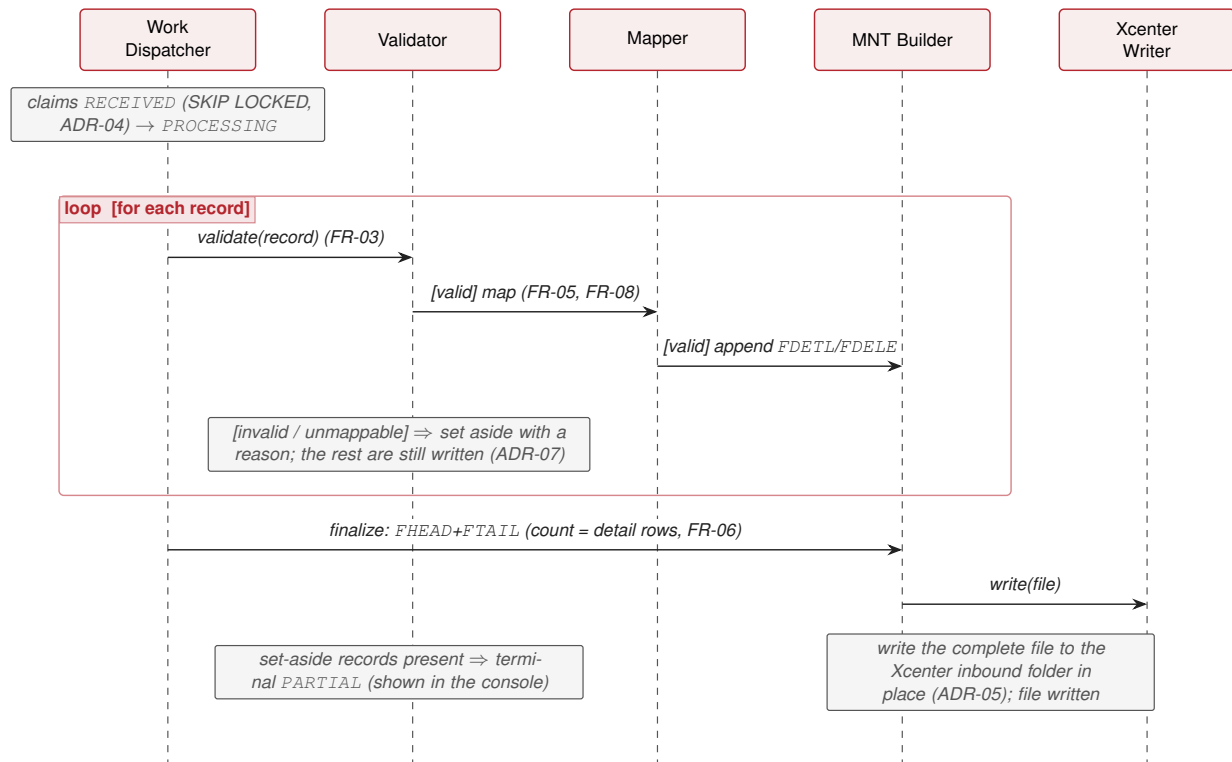


Figure 13: S3 – partial batch (records set aside, file still written).

Outcome. `PARTIAL`. The set-aside records and their reasons are visible in the console event detail. If set-aside records exceed the configured threshold (default 20%, ADR-07) the run aborts instead of finishing `PARTIAL`.

6.5. S4 – Write failure and retry [exception flow]

Trigger. The MNT is built but the write to the Xcenter inbound folder fails (disk full, permission, or the share is unavailable). Because Xstore ingests only complete files (its pickup contract, ADR-05), an interrupted write is never picked up; the batch moves to `PENDING_WRITE` and the Retry Scheduler re-drives it with bounded exponential backoff (FR-12, NFR-04), rewriting the file until it succeeds.

Only after the retry budget is exhausted does the batch fail and Ops get alerted.

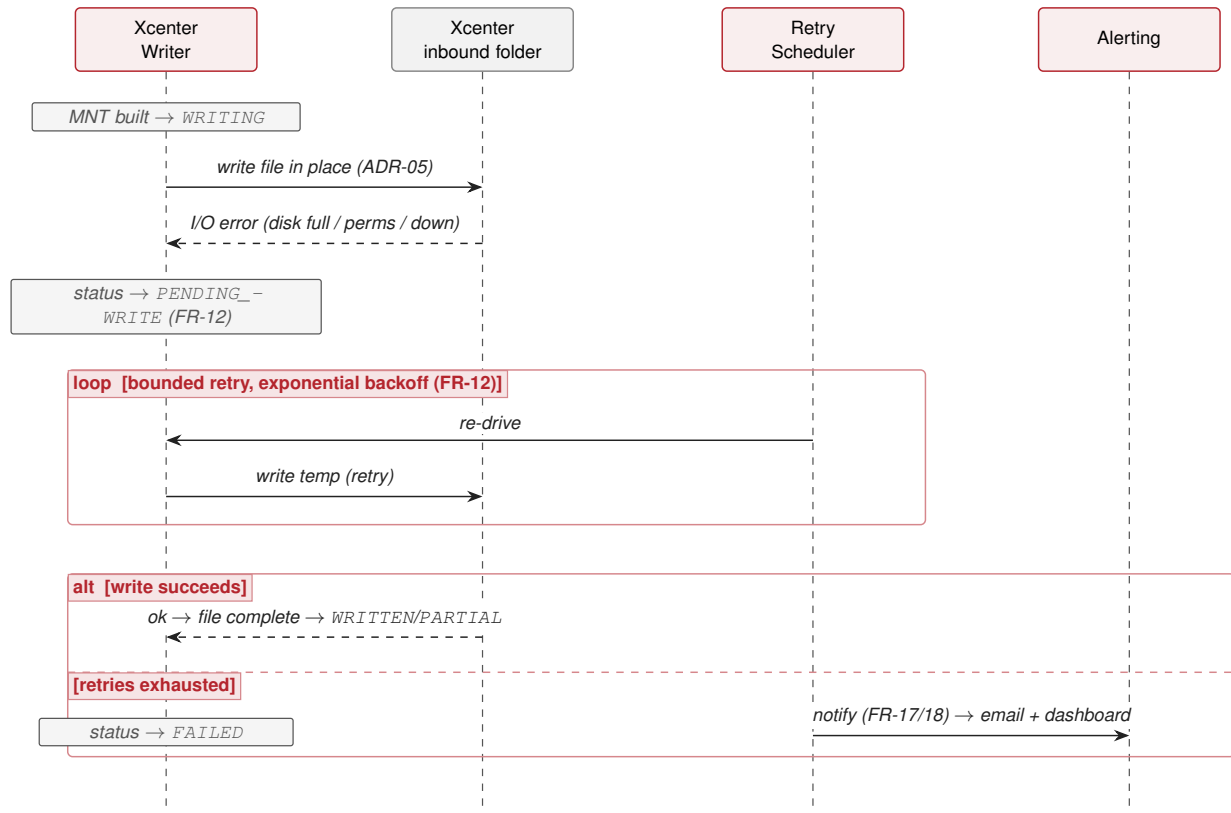


Figure 14: S4 – write failure, bounded retry, then success or alert.

Exception handling. A transient write error is absorbed by the retry loop and usually resolves without anyone noticing. A persistent failure ends in `FAILED` and a single alert – on the dashboard and by email to the Ops group – so Ops can act; the unwritten batch is preserved in the database – nothing is lost.

Automatic retries do not resume on their own: once the cause is fixed, an operator re-drives the batch from the console (POST `/api/v1/events/{id}/retry`, administrator role; Section 7.3), which returns it to `PENDING_WRITE` for the Retry Scheduler.

6.6. S5 – Instance failover [exception flow]

Trigger. The application instance processing a batch crashes or is restarted mid-run. Claiming is a short transaction that commits `PROCESSING` together with an ownership lease *before* any work starts (ADR-04), so a crash does *not* roll the batch back – instead the owner simply stops renewing its lease.

A periodic reaper on the surviving instance detects the expired lease, resets the batch to `RECEIVED`, and it is re-claimed with `SELECT ...FOR UPDATE SKIP LOCKED`. Because only an *expired* lease is ever reset and claiming uses `SKIP LOCKED`, a single processor holds the batch at any time and there is never a double write.

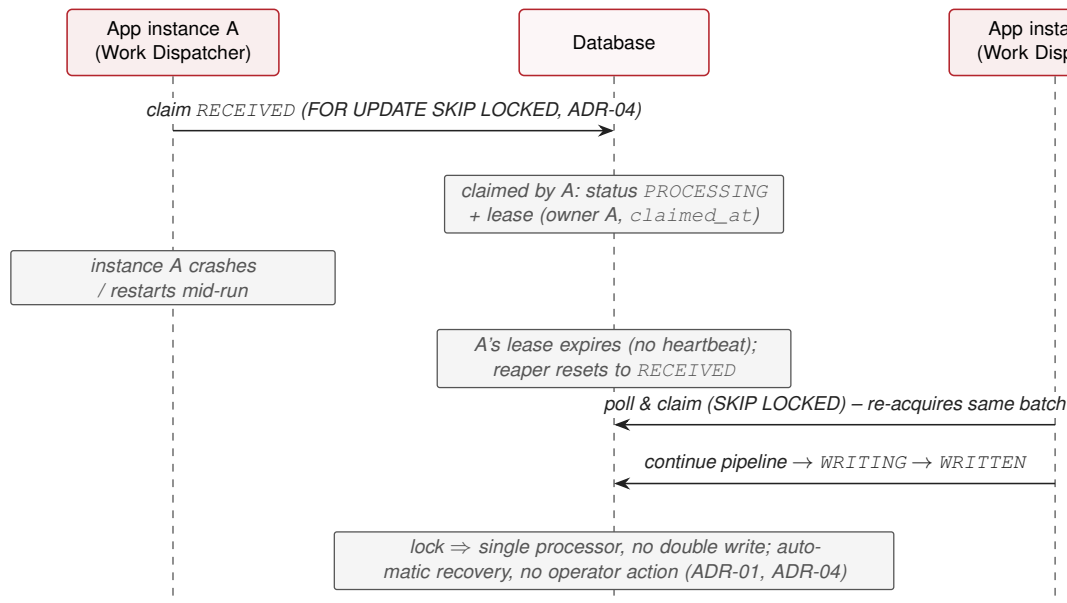


Figure 15: S5 – instance failover during processing (no double write).

Outcome. The batch resumes on the surviving instance and reaches its normal terminal state. Both instances stay active throughout (active/active); failover needs no manual step.

7. API Design

7.1. Conventions

All endpoints are served over TLS and versioned under `/api/v1`. Two kinds of caller use the API, authenticated differently:

- *HQ (machine-to-machine)* – calls only the intake endpoint, authenticated per request with an API key, an HMAC-SHA256 body signature, and an IP allowlist (NFR-02).
- *Operations staff (console)* – call the read and configuration endpoints through the web UI, authenticated as operator users; configuration changes additionally require an administrator role.

The intake endpoint is **idempotent**: re-sending the same `batch_id+version` returns 409 as a no-op, while a higher `version` of the same `batch_id` is accepted as a new batch that supersedes the previous one (ADR-06). Supersede here means the new version produces a fresh corrected file that overwrites the prices downstream; it does *not* un-write or roll back a file already written for an earlier version (the system cannot reach into Xstore) – a correction is therefore expressed as a later, higher-version batch. Request and response bodies are JSON (`application/json`, UTF-8) and all timestamps are ISO-8601 with timezone. Every error uses the one shape in Section 7.4.

List endpoints (events, logs) accept `page` (0-based) and `size` query parameters plus resource-specific filters, and return a paged envelope:

```
{ "items": [ /* ... */ ], "page": 0, "size": 50, "total": 320 }
```

7.2. Intake endpoint

POST `/api/v1/price-events` – the single entry point HQ calls each night. It accepts one price **batch** (an envelope carrying many records; one record becomes one MNT detail row), authenticates and structurally validates the envelope, persists it as `RECEIVED`, and returns `202` immediately; per-record validation, mapping, and file writing then happen asynchronously (ADR-02).

(The envelope/record structure below is the agreed HQ contract and the design baseline.)

Request headers.

Header	Req.	Purpose
Content-Type	yes	<code>application/json</code> , UTF-8.
X-API-Key	yes	The HQ client's API key, identifying the caller.
X-Signature	yes	HMAC-SHA256 of the raw request body (hex), keyed with the shared secret; proves integrity and authenticity.
X-Timestamp	yes	Epoch seconds when the request was signed; bounds replay (Section 7.5).

Table 14: Intake request headers.

Request body (batch envelope).

```
{
  "batch_id": "2026-06-20-nightly",
  "version": 1,
```

```

"generated_at": "2026-06-19T22:00:00+07:00",
"records": [
  { "change_id": "alf3c9e2", "version": 3,
    "item_id": "SKU123", "store_id_or_zone": "ZONE_NORTH",
    "price": 99000, "currency": "VND",
    "effective_start": "2026-06-20", "effective_end": "2026-06-30",
    "change_type": "update" },
  { "change_id": "b7c20f55", "version": 1,
    "item_id": "SKU124", "store_id_or_zone": "STORE_001",
    "price": 49000, "change_type": "new" }
]
}

```

Envelope fields.

Field	Type	Req.	Meaning
batch_id	string	yes	Unique id of the nightly batch; with version it forms the batch idempotency key.
version	integer	yes	Batch version; a higher value supersedes the same batch_id (ADR-06).
generated_at	timestamp	yes	When HQ produced the batch.
records	array	yes	The price-change records; must be non-empty (else 422).

Table 15: Price-batch envelope fields.

Record fields (each record becomes one MNT detail row).

Field	Type	Req.	Meaning
change_id	string	yes	Unique id of this price change; with <code>version</code> it forms the record-level idempotency key.
version	integer	yes	Record version; the latest version wins (ADR-06).
item_id	string	yes	HQ item / SKU code; mapped to the Xstore item code.
store_id_or_zone	string	yes	HQ location – a single store or a zone / region; mapped to Xstore <code>LOCATION</code> + <code>LOC_TYPE</code> .
price	number	yes*	New price; written with a decimal point, no thousands separator, scale per currency (VND → 0 decimals), configurable. <i>*Omitted when <code>change_type=delete</code>.</i>
currency	string	no	ISO-4217 code; defaults to the configured currency if absent.
effective_start	date	no	When the price takes effect; defaults to <code>D+1</code> if absent (FR-08), where <code>D</code> is the batch business date derived from <code>generated_at</code> in the configured timezone (so a near-midnight run does not drift a day).
effective_end	date	no	When the price stops applying. If absent the price is open-ended – written with no end date and stays in effect until a later change supersedes it, with no auto-revert; auto-revert (handled by Xstore, out of scope) applies only when an end date is present.
change_type	enum	yes	<code>new / update</code> → <code>FDETL</code> (create/modify); <code>delete</code> → <code>FDELE</code> .

Table 16: Price-record fields.

Responses.

Code	Meaning	When
202	Accepted	Batch authenticated and persisted (status <code>RECEIVED</code>); processed asynchronously.
400	Bad Request	Malformed JSON or missing required envelope fields.
401	Unauthorized	Missing/invalid API key or HMAC signature.
403	Forbidden	Source IP not on the allowlist.
409	Conflict	Same <code>batch_id+version</code> already received (idempotent no-op).
422	Unprocessable	Envelope well-formed but fails a batch-level rule (e.g. empty records).
429	Too Many Requests	Request-rate limit exceeded; retry after the window (Section 7.5).

Table 17: Intake responses.

202 response. Confirms receipt only – not per-record validity.

```
{ "batch_id": "2026-06-20-nightly", "version": 1,
  "status": "RECEIVED", "received_records": 2 }
```

Per-record validation happens asynchronously; a record that later fails is set aside with a reason and surfaced in the console (status `PARTIAL`), not rejected at intake.

7.3. Console and administration endpoints

Used by operations staff through the web UI – operator authentication, with configuration changes (PUT) and the manual re-drive restricted to an administrator role. The summary table indexes the endpoints; each is then specified in full below. Unless stated otherwise, list endpoints follow the paging convention of Section 7.1, 401 `UNAUTHENTICATED` applies to every endpoint when the operator session is missing or expired, and every error body uses the shape of Section 7.4.

Endpoint	What it does (params / returns)
GET <code>/api/v1/events</code>	Lists batches. Filters: <code>status</code> , <code>from/to</code> date; paged. Returns <code>id</code> , <code>version</code> , <code>status</code> , <code>record counts</code> , and <code>timestamps</code> per batch.
GET <code>/api/v1/events/{id}</code>	Returns one batch in full: envelope header, each record's outcome, and the reason for every set-aside record.
POST <code>/api/v1/events/{id}/retry</code>	Re-drives a <code>FAILED</code> batch: returns it to <code>PENDING_WRITE</code> for the Retry Scheduler to re-attempt. Idempotent (a no-op if the batch is not <code>FAILED</code>); administrator role.
GET <code>/api/v1/logs</code>	Per-step audit trail (received / validated / transformed / written / retry) for a batch or time range; filterable and paged.
GET <code>/api/v1/dashboard/metrics</code>	Daily counters (received / written / partial / failed) and the list of items needing attention.
GET, PUT <code>/api/v1/config/connections</code>	Reads or updates the HQ-inbound and Xcenter-outbound connection settings (folders, endpoints, credential references). PUT requires the administrator role.
GET, PUT <code>/api/v1/config/mappings</code>	Reads or updates the field mappings, value maps, and defaults the Mapper applies. PUT requires the administrator role.
GET <code>/actuator/health</code>	Minimal liveness / readiness probe for the load balancer; unauthenticated by design and exposes no internal detail. All other <code>/actuator/*</code> endpoints are disabled or require operator authentication.

Table 18: Console and administration endpoints.

GET /api/v1/events – lists batches for the event-history view (FR-16).

Param	Type	Default	Meaning
status	enum	–	Filter by batch state (Figure 4): RECEIVED ... FAILED.
from, to	ISO-8601 date	–	Bound the query on received_at.
page, size	int	0, 50	Paging (Section 7.1); size capped at 200.

Response 200 – paged envelope; one item per batch version:

```
{ "items": [
  { "batch_id": "2026-06-20-nightly", "version": 1, "status": "WRITTEN",
    "received_at": "2026-06-19T22:00:04+07:00",
    "generated_at": "2026-06-19T22:00:00+07:00",
    "item_count": 41230, "written_count": 41230, "set_aside_count": 0,
    "file": "pricesync_2026-06-20-nightly_v1_20260620T011502.mnt" } ],
  "page": 0, "size": 50, "total": 320 }
```

Errors: 400 MALFORMED_REQUEST (unknown status value, malformed date), 401.

GET /api/v1/events/{id} – one batch in full: header, file metadata, and per-record outcomes (FR-16). {id} is the batch_id; version defaults to the latest received version. Because a batch can carry tens of thousands of records, the records array is itself paged and filterable by outcome (WRITTEN / SET_ASIDE / SUPERSEDED) – the console uses outcome=SET_ASIDE to show only the records needing follow-up (ADR-07).

```
{ "batch_id": "2026-06-20-nightly", "version": 2, "status": "PARTIAL",
  "received_at": "2026-06-20T02:10:11+07:00",
  "written_at": "2026-06-20T02:14:03+07:00",
  "item_count": 41230, "written_count": 41198, "set_aside_count": 32,
  "file": { "name": "pricesync_2026-06-20-nightly_v2_20260620T021403.mnt",
    "record_count": 41198, "checksum": "sha256:9f2c..." },
  "records": { "items": [
    { "change_id": "a1f3c9e2", "version": 3, "item_id": "SKU123",
      "store_id_or_zone": "ZONE_NORTH",
      "outcome": "SET_ASIDE", "reason": "UNKNOWN_LOCATION" } ],
    "page": 0, "size": 50, "total": 41230 } }
```

Errors: 404 NOT_FOUND (unknown batch_id/version, or purged after retention – NFR-08), 400, 401.

POST /api/v1/events/{id}/retry – administrator re-drive of a FAILED batch (S4). The optional body {"reason": "share restored"} is recorded in the audit trail (FR-14). On a FAILED batch it returns the batch to PENDING_WRITE for the Retry Scheduler; on any other state it is an idempotent no-op.

Code	error	When
202	–	Batch was FAILED; now PENDING_WRITE, re-armed for retry.
200	–	No-op: batch is not FAILED; current status returned, nothing changed.
403	ROLE_REQUIRED	Caller is an operator without the administrator role.
404	NOT_FOUND	Unknown batch_id/version.

GET /api/v1/logs – the per-step audit trail (FR-14). Filters: batch_id (+version), step (received/validated/transformed/written/retry), from/to; paged.

```
{ "items": [
  { "ts": "2026-06-20T02:14:03+07:00",
    "batch_id": "2026-06-20-nightly", "version": 2,
    "step": "written", "outcome": "ok",
    "message": "wrote pricesync_..._v2_20260620T021403.mnt" } ],
  "page": 0, "size": 50, "total": 6 }
```

Errors: 400, 401.

GET /api/v1/dashboard/metrics – daily counters behind the dashboard (FR-15). date defaults to today in the configured timezone.

```
{ "date": "2026-06-20",
  "received": 1, "written": 0, "partial": 1, "failed": 0, "pending_write": 0,
  "set_aside_records": 32,
  "last_batch": { "batch_id": "2026-06-20-nightly", "version": 2,
    "status": "PARTIAL" },
  "attention": [ { "batch_id": "2026-06-20-nightly", "version": 2,
    "status": "PARTIAL", "reason": "32 records set aside" } ] }
```

Errors: 401.

GET, PUT /api/v1/config/connections – the HQ-intake and Xcenter-output connection settings (ADR-08). Secrets never appear here: the document carries `authRef` references resolved from the secret store at runtime, and **PUT** accepts only references – rotation happens in the store, not through this API (ADR-11).

```
{ "version": 7, "updated_at": "2026-06-18T09:31:00+07:00",
  "updated_by": "ops.admin",
  "hq_intake": { "ip_allowlist": ["203.0.113.10"],
    "api_key_ref": "kv/pricesync/hq-api-key",
    "hmac_secret_ref": "kv/pricesync/hq-hmac",
    "rate_limit_per_min": 5, "replay_skew_min": 5 },
  "xcenter": { "inbound_path": "/mnt/xcenter/inbound",
    "pickup_mode": "in_place",
    "filename_pattern": "pricesync_<batch_id>_v<version>_<ts>.mnt" } }
```

PUT (administrator) replaces the whole document and must carry the `version` it was read at – an optimistic lock, so two concurrent editors cannot silently overwrite each other. The new values are applied from the start of the next batch (the per-batch cache reload of Section 5.4); the change is audited.

Code	error	When
200	–	Accepted; response carries the new <code>version</code> .
403	ROLE_REQUIRED	PUT without the administrator role.
409	CONFIG_CONFLICT	Stale <code>version</code> : reload and re-apply the edit.
422	VALIDATION_FAILED	Empty <code>inbound_path</code> , malformed <code>filename_pattern</code> , unknown <code>pickup_mode</code> , raw secret value supplied.

GET, PUT /api/v1/config/mappings – the rules the Mapper and MNT Builder apply (FR-05, ADR-08): field-to-column map, value maps, defaults, and output format.

```
{ "version": 12, "updated_at": "2026-06-15T14:02:00+07:00",
  "field_map": [ { "json_field": "item_id", "mnt_column": "ITEM_ID" } ],
  "value_maps": { "change_type": { "new": "FDETL", "update": "FDETL",
    "delete": "FDELE" },
    "loc_type": { "store": "S", "zone": "Z" } },
  "defaults": { "currency": "VND", "eff_start": "D+1" },
  "output": { "delimiter": ",", "quoting": "rfc4180",
    "encoding": "UTF-8" } }
```

PUT follows the same administrator / optimistic-lock / next-batch semantics as `/config/connections`. 422 `VALIDATION_FAILED` lists every rejected entry – an `mnt_column` unknown to the mapping profile (Host Interface Guide), a duplicate `json_field`, or a mandatory column left unmapped – so one round trip surfaces all problems.

GET /actuator/health – liveness / readiness probe for the load balancer: 200 {"status": "UP"} or 503 {"status": "DOWN"}. Unauthenticated by design and exposes no internal detail; all other /actuator/* endpoints are disabled or require operator authentication.

7.4. Error catalog

All errors share one shape so callers can handle them uniformly: `error` is a stable machine-readable code, `message` a human-readable detail, `batch_id+version` identify the offending batch when applicable, and `ts` is the server timestamp.

```
{ "error": "BATCH_DUPLICATE",
  "message": "batch_id+version already received",
  "batch_id": "2026-06-20-nightly", "version": 1,
  "ts": "2026-06-19T22:00:05+07:00" }
```

error	HTTP	Caller action
MALFORMED_REQUEST	400	Fix the payload/JSON and resend.
UNAUTHENTICATED	401	Check API key / HMAC signature and clock.
IP_NOT_ALLOWED	403	Send from an allowlisted address.
ROLE_REQUIRED	403	Console write endpoints: sign in with an administrator account.
NOT_FOUND	404	Unknown batch_id/version; very old batches may have been purged (NFR-08).
BATCH_DUPLICATE	409	None - already accepted (idempotent).
CONFIG_CONFLICT	409	Console PUT: reload the configuration and re-apply on the latest version.
EMPTY_BATCH	422	Resend with a non-empty <code>records</code> array.
VALIDATION_FAILED	422	Console PUT: fix the fields listed in <code>message</code> and resend.
RATE_LIMITED	429	Back off and retry after the rate-limit window.

Table 19: Error catalogue: codes, HTTP status, and caller action.

7.5. Rate limiting and replay protection

The intake endpoint accepts a single nightly batch, so a low request-rate limit is sufficient and protects against accidental floods; a request over the limit is rejected with 429 (`RATE_LIMITED`). The limit is enforced at the reverse proxy / edge (ADR-12) so it is *global*, not multiplied by the number of app instances; a sensible default is a handful of requests per minute per client [*exact value is configuration*].

Replay is bounded by the signed X-Timestamp: requests whose timestamp falls outside a small clock-skew window (default ± 5 minutes, configurable) are rejected, and this together with batch-level idempotency means a captured request cannot be re-applied.

8. Test Plan

This section defines how the design is verified. It sets the test levels and tooling, lists the functional test cases that realise the requirements digest (Section 2) and the runtime scenarios S1–S5 (Section 6), and closes with the non-functional cases behind the NFR targets and the release exit criteria.

Cases are numbered TC-*nn* (functional) and TC-*Nn* (non-functional). Appendix A maps each requirement to the scenario that verifies it; the tables below make those scenarios executable.

8.1. Test levels and tooling

Level	Scope and focus	Tooling
Unit	Validator rules, Mapper transforms, MNT Builder formatting, idempotency-key logic – pure, fast, no I/O.	JUnit 5, AssertJ
Integration	Repositories, SKIP LOCKED claiming, lease/reaper, Flyway schema – against a real database.	Spring Boot Test, Testcontainers (PostgreSQL 14)
API / contract	Intake endpoint: authentication matrix, error-catalog shape, console endpoints and roles.	MockMvc / WebClient
End-to-end scenario	One nightly run through the real pipeline; S1–S5 branch coverage; the produced file is inspected on a stand-in share.	Full Spring context, @TempDir share, GreenMail (alert email)
Non-functional	NFR-01 volume run, failover drill, security checks, retention purge.	Synthetic batch generator, continuous-integration (CI) jobs

Table 20: Test levels, scope, and tooling.

Unit, integration, and API suites run in CI (Maven) on every commit; the end-to-end and non-functional suites also run nightly against a production-like configuration.

8.2. Functional test cases

Unless a case states otherwise, requests are authenticated and sent from an allowlisted IP. Assertions cover four surfaces: the HTTP response, the batch status in the database, the audit trail, and the presence or absence of a file on the share.

Intake and security.

ID	Covers	Test case	Expected result
TC-01	FR-01, FR-02	POST a valid batch with correct API key, HMAC, and allowlisted IP.	202; batch persisted RECEIVED with parsed records; first audit entry written.
TC-02	FR-02, S1	Missing or wrong HMAC signature (or API key).	401 UNAUTHENTICATED; nothing persisted.
TC-03	FR-02, S1	Request from a non-allowlisted source IP.	403 IP_NOT_ALLOWED; nothing persisted.
TC-04	S1	Malformed JSON or missing envelope field; empty records array.	400 MALFORMED_REQUEST; 422 EMPTY_BATCH; nothing persisted.
TC-05	§7.5	X-Timestamp outside the ± 5 min skew window; replay of a captured request; burst over the rate limit.	Stale/replayed request rejected; over-limit requests get 429 RATE_LIMITED.

Table 21: Test cases – intake and security.

Idempotency and versioning (ADR-06).

ID	Covers	Test case	Expected result
TC-06	FR-13, S2	Re-send the same batch_id+version.	409 BATCH_DUPLICATE; original batch untouched; no second file.
TC-07	FR-13	Send a higher version of the same batch_id.	Accepted as a superseding batch; a fresh file is produced under a distinct name.
TC-08	FR-13	Within a batch, a record with a higher change_id version; then a lower/equal one.	Higher version supersedes the stored record; lower or equal is ignored.
TC-09	ADR-06	Two concurrent inserts of the same batch key (race).	Second insert fails on the UNIQUE constraint; exactly one row survives.

Table 22: Test cases – idempotency and versioning.

Validation, mapping, and file build.

ID	Covers	Test case	Expected result
TC-10	FR-03, S3	Batch mixing valid records with invalid ones ($\text{price} \leq 0$, bad currency, start after end).	Invalid records set aside with reasons; valid rows written; batch ends PARTIAL.
TC-11	FR-05	Record with an unknown item or location code.	Treated as <i>unmappable</i> : set aside with a reason, not a hard format error.
TC-12	ADR-07	More than 20% of records invalid, or no valid record remains.	Run aborts instead of finishing PARTIAL; no file written.
TC-13	FR-06, NFR-03	Golden-file check of a produced MNT against the mapping profile.	FHEAD, one FDETL/FDELE per record, FTAIL; trailer count equals detail rows; delimiter/quoting per profile.
TC-14	FR-07, FR-08	Records with and without effective dates, including a batch generated near midnight.	Dates carried through; absent start $\Rightarrow$ D+1 of the batch business date (timezone-safe); absent end $\Rightarrow$ open-ended.
TC-15	FR-05	change_type new/update/delete; store vs zone locations.	FDETL vs FDELE chosen correctly; LOC_TYPE derived per record (S/Z).

Table 23: Test cases – validation, mapping, and file build.

Write path and reliability.

ID	Covers	Test case	Expected result
TC-16	FR-09, ADR-05, S0	Successful write to the stand-in share.	Complete file written in place as <code>pricesync_<batch_id>_v<version>_<timestamp>.mnt</code> with a correct trailer count; status <code>WRITTEN</code> .
TC-17	FR-12, S4	Make the inbound folder unwritable, then restore it mid-budget.	Batch parks at <code>PENDING_WRITE</code> ; retries back off (30 s doubling, cap 10 min); on success a complete file is written; a failed attempt is retried and rewritten.
TC-18	NFR-04, S4	Keep the folder unwritable through all 6 attempts.	Status <code>FAILED</code> ; exactly one deduplicated alert (dashboard + email); batch preserved in the database.
TC-19	§7.3	<code>POST /api/v1/events/{id}/retry</code> as administrator on a <code>FAILED</code> batch; then on a non- <code>FAILED</code> batch; then as plain operator.	Returns the batch to <code>PENDING_WRITE</code> ; no-op on non- <code>FAILED</code> ; denied without the administrator role.
TC-20	ADR-04, S5	Kill the owning instance mid-run (after claim, before write).	Lease expires; reaper resets to <code>RECEIVED</code> ; the surviving instance re-claims and completes; exactly one file, no double write.
TC-21	ADR-04	Both instances poll simultaneously for the same <code>RECEIVED</code> batch.	<code>SKIP_LOCKED</code> lets exactly one claim it; the other skips without blocking.

Table 24: Test cases – write path and reliability.

Operations and observability.

ID	Covers	Test case	Expected result
TC-22	FR-11, FR-14	Run a batch end to end, then query <code>GET /api/v1/logs</code> by batch and by time range.	One audit entry per pipeline step (received / validated / transformed / written / retry), filterable and paged.
TC-23	FR-15, FR-16	<code>GET /api/v1/events</code> and <code>/events/{id}</code> after a <code>PARTIAL</code> run; dashboard metrics.	Statuses, record counts, and every set-aside reason returned; daily counters correct.
TC-24	FR-17, FR-18	Let the 05:00 cut-off pass with no batch received.	Missing-batch watchdog raises one alert on both channels (dashboard + email).
TC-25	ADR-08, NFR-09	Edit a mapping rule and a connection via console <code>PUT</code> (administrator), then run the next batch.	Change applied at next batch start without redeploy; picked up by both instances within the TTL; edit audited.

Table 25: Test cases – operations and observability.

8.3. Non-functional test cases

ID	Covers	Test case	Expected result
TC-N1	NFR-01	Synthetic batch at the ceiling: 250 stores / 75 000+ items, run on reference hardware.	Completes well within the nightly window; runtime recorded per release.
TC-N2	NFR-01	Same volume run with heap monitoring.	Memory stays bounded (streamed build); no full-batch buffering.
TC-N3	NFR-06	Failover drill (TC-20) while intake receives requests.	Intake stays reachable throughout; the nightly run still completes.
TC-N4	NFR-02	Scan database dump, logs, and audit entries for secret material; verify TLS at the proxy.	No plaintext secret anywhere (<code>authRef</code> only); TLS 1.2+ enforced at the edge.
TC-N5	NFR-08	Age terminal batches past the retention window and run the purge job.	Batches, records, file metadata, and audit older than the window (≥ 2 months) removed; newer data intact.
TC-N6	NFR-03, NFR-05	Property-based check over generated batches: every produced file re-parsed.	Trailer count always equals detail rows; file content reflects exactly the validated records.

Table 26: Non-functional test cases.

8.4. Entry and exit criteria

Entry. Schema migrated (Flyway); mapping profile, value maps, and connections seeded; stand-in share mounted and writable; secret store reachable.

Exit (per release).

- All functional cases (TC-01..TC-25) green in CI; the S1–S5 end-to-end suite green, including the failover drill.
- Non-functional runs within targets: TC-N1 within the window, TC-N2 bounded memory.
- Line coverage $\geq 80\%$ on the Validator, Mapper, MNT Builder, and Work Dispatcher.
- No unresolved critical static-analysis or dependency findings.

Regression. Unit, integration, and API suites on every commit; end-to-end and failover nightly; the volume run (TC-N1) before each release.

A. Requirements traceability

Each URD requirement is mapped to the design element that satisfies it and to how it is verified. Verification today is by the quality-attribute scenarios (Section 2.3), the runtime scenarios S0–S5 (Section 6), and design review, realised as the automated test cases TC-01 . . 25 and TC-N1 . . N6 in Section 8.

Req.	Satisfied by (design element)	Verified by
FR-01	Intake Controller <code>POST /price-events</code> (Section 6.2); ADR-02	S0; Perf/Security scenarios
FR-02	Security filter chain (API key / HMAC / IP), ADR-12; NFR-02	S1; Security scenario
FR-03	Validator (Section 5.3); ADR-07	S3
FR-04	Delta intake; <code>PriceRecord</code> model; Intake Service	Review
FR-05	Mapper / Transformer (Section 5.3); ADR-08	S3; Modifiability scenario
FR-06	<code>XstoreMntBuilder</code> , <code>FHEAD/FDETL/FDELE/FTAIL count</code> (Section 5.3)	NFR-03; S0 step 5
FR-07	Mapper date fields; record fields (Section 6.2)	Review
FR-08	Mapper default <code>D+1</code> (Section 5.3, Section 6.2)	Review
FR-09	Xcenter Writer, in-place write, ADR-05	S0; S4
FR-10	Daily processing flow (Section 3.4)	S0
FR-11	Batch lifecycle (Section 3.5); <code>AuditEvent</code>	S0; all S-scenarios
FR-12	Retry Scheduler, ADR-04/05; NFR-04	S4; Reliability scenario
FR-13	Idempotency Manager, ADR-06	S2
FR-14	<code>AuditEvent</code> ; <code>GET /logs</code> (Section 6.3)	Review
FR-15	Console (Section 5.4); ADR-08; dashboard metrics	Review
FR-16	<code>GET /events</code> , <code>GET /events/{id}</code> (Section 6.3)	Review
FR-17/18	Alerting (Section 5.4); email + dashboard	S4
NFR-01	Streaming builder; performance budget (Section 2.3)	Perf/Scalability scenarios
NFR-02	ADR-11 (secrets), ADR-12 (TLS), filter chain	Security scenario; S1
NFR-03	Trailer count == detail rows; complete file, pickup contract (ADR-05)	Review; S3
NFR-04	ADR-04 (lease/reaper), Retry Scheduler	Reliability scenario; S4/S5
NFR-05	Validate → map → build; <code>FTAIL</code> self-check	Review
NFR-06	Two instances; ADR-04 claim + reaper	Availability scenario; S5
NFR-07	Actuator/Micrometer, structured logs, per-event status	Review
NFR-08	<code>AuditEvent</code> ; Retention/Purge; Appendix B	Review
NFR-09	ADR-09 output seam; ADR-08 config	Modifiability scenario

Table 27: Requirements traceability matrix.

B. Persistence schema and key indexes

A logical sketch of the core tables and the constraints/indexes the design depends on. Exact column types and migrations (Flyway) are defined in the codebase; this fixes the invariants the rest of the document relies on.

```

-- Idempotency: a re-sent (batch_id, version) is rejected (ADR-06)
ALTER TABLE price_batch
  ADD CONSTRAINT uq_batch UNIQUE (batch_id, version);

-- Lease columns for short-claim + reaper recovery (ADR-04)
-- status, owner_instance, claimed_at on price_batch

-- Claim query feeder: only rows the workers poll for
CREATE INDEX ix_batch_claimable ON price_batch (status)
  WHERE status IN ('RECEIVED', 'PENDING_WRITE');

-- Reaper feeder: stale leases in in-flight states
CREATE INDEX ix_batch_lease ON price_batch (claimed_at)
  WHERE status IN ('PROCESSING', 'WRITING');

-- Record-level idempotency: latest (change_id, version) wins (ADR-06)
ALTER TABLE price_record
  ADD CONSTRAINT uq_change UNIQUE (change_id, version);
CREATE INDEX ix_record_batch ON price_record (batch_id);

-- Audit + retention support
CREATE INDEX ix_audit_batch_ts ON audit_event (batch_id, ts);

```

Notes. The two UNIQUE constraints are what make idempotency safe under concurrent duplicates – a second insert fails at the database rather than racing in the application (Section 5.2).

The two partial indexes keep the Work Dispatcher and Retry Scheduler claim queries cheap by indexing only the small set of rows actually polled, which is what keeps FOR UPDATE SKIP LOCKED fast at scale (NFR-01). The raw HQ payload is stored as `jsonb` for audit/replay.

Retention: a purge job (Section 5.4) deletes terminal batches and their records / audit older than the configured window (≥ 2 months, NFR-08); the window must exceed any plausible HQ re-send delay so a legitimate re-send is never re-processed as new once its idempotency keys are purged.